ใบอนุมัติโครงงานปริญญานิพนธ์
สาขาวิชาวิทยาการคอมพิวเตอร์ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

ชื่อปริญญานิพนธ์ การจำแนกยุคสมัยประติมากรรมลวดลายปูนปั้นประดับโบราณสถานไทยด้วยการเรียนรู้เชิงลึก
A Period Classification of Sculptures and Stucco Patterns Decorating Thai Historical Sites using Deep Learning

ผู้จัดทำ นาย ณภัทร ปัทมโรจน์

ปีการศึกษา 2567

โครงงานปริญญานิพนธ์นี้ได้รับการอนุมัติให้เป็นส่วนหนึ่งของการศึกษาตาม
หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์
ภาควิชาคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

..... ประธานกรรมการสอบ
(อาจารย์ ดร.ณัฐพงศ์ แตงจุ้ย)

..... กรรมการสอบ
(อาจารย์ ดร.ภูริวัจน์ วรวิชัยพัฒน์)

..... อาจารย์ที่ปรึกษาและกรรมการสอบ
(ผู้ช่วยศาสตราจารย์โอภาส วงษ์ทวีทรัพย์)

การจำแนกยุคสมัยประติมากรรมลวดลายปูนปั้นประดับโบราณสถานไทยด้วยการเรียนรู้เชิงลึก

A Period Classification of Sculptures and Stucco Patterns Decorating Thai Historical Sites using Deep Learning

นายณภัทร ปัทมโรจน์

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาในหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาวิชาวิทยาการคอมพิวเตอร์ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร
ปีการศึกษา 2567

ชื่อปริญญานิพนธ์ การจำแนกยุคสมัยประติมากรรมลวดลายปูนปั้นประดับโบราณสถานไทยด้วยการเรียนรู้เชิงลึก
A Period Classification of Sculptures and Stucco Patterns Decorating Thai Historical Sites using Deep Learning

ผู้จัดทำ นาย ณภัทร ปัทมโรจน์
อาจารย์ที่ปรึกษา ผู้ช่วยศาสตราจารย์โอภาส วงษ์ทวีทรัพย์

ชื่อปริญญา วิทยาศาสตรบัณฑิต (วิทยาการคอมพิวเตอร์)
ปีการศึกษา 2567

บทคัดย่อ

ประติมากรรมตามพจนานุกรมราชบัณฑิตยสถานมีความหมายว่า ศิลปะสาขาหนึ่งในจำพวกวิจิตรศิลป์เกี่ยวกับการแกะสลักไม้ หินอ่อน โลหะ ให้เป็นรูปหรือลวดลายต่าง ๆ ในประเทศไทยนิยมนำปูนมาปั้นเป็นลวดลายเพื่อประดับศาสนสถานหรือโบราณสถานต่าง ๆ ลวดลายปูนปั้นในโบราณสถานของไทยเป็นสมบัติทางวัฒนธรรมอันล้ำค่าที่สะท้อนให้เห็นถึงประวัติศาสตร์ ศิลปะ และวัฒนธรรมของชาติไทยมาช้านาน ประติมากรรมลวดลายปูนปั้นโบราณเหล่านี้มีอายุตั้งแต่สมัยทวารวดีจนถึงสมัยรัตนโกสินทร์แต่ละยุคสมัยมีรูปแบบและลักษณะเฉพาะตัวที่บ่งบอกถึงพัฒนาการทางศิลปะและวัฒนธรรมในช่วงเวลานั้น ๆ

ในโครงงานปริญญานิพนธ์นี้นำเสนอวิธีการจำแนกยุคสมัยของประติมากรรมลวดลายปูนปั้นประดับโบราณสถานไทยโดยใช้เทคโนโลยีการเรียนรู้เชิงลึกซึ่งเป็นเครื่องมือที่มีประสิทธิภาพในการประมวลผลข้อมูลเชิงภาพและจำแนกความแตกต่างของลวดลายทางศิลปะ ในโครงงานนี้ได้ทำการเก็บรวบรวมภาพถ่ายของลวดลายปูนปั้นจากโบราณสถานในยุคต่าง ๆ ของไทยเพื่อใช้เป็นข้อมูลในการฝึกสอนโมเดล โดยโมเดลที่นำมาใช้นี้เป็นโครงข่ายประสาทเทียมแบบคอนโวลูชันแบบเปลี่ยนขนาดเคอร์เนลในการทดลองทั้งหมด 3 ขนาดคือ 5x5, 11x11 และ 15x15 และ เรสเน็ต50 และแยกข้อมูลดาตาเซ็ท 2 แบบคือภาพสี และภาพขาวเทา มีการเชื่อมต่อกับผู้ใช้งานผ่านฟลาสก์

โดยส่วนการทดลองด้วยโมเดลโครงข่ายประสาทเทียมแบบคอนโวลูชันมีผลลัพธ์ดังนี้ ความแม่นยำเฉลี่ยโดยรวมของโมเดลภาพสีอยู่ที่ 92.50% ที่เคอร์เนลขนาด15x15 และความแม่นยำเฉลี่ยโดยรวมของโมเดลภาพขาวเทาอยู่ที่ 86.24% ที่เคอร์เนลขนาด11x11 โดยในส่วนของการแสดงผลของเว็บแอปพลิเคชัน ระบบจะทำการจำแนกลวดลายปูนปั้นจากโบราณสถานที่ได้รับออกเป็น 6 คลาสได้แก่ ยุคสมัยสุโขทัย, อยุธยาตอนต้น, อยุธยาตอนกลาง, อยุธยาตอนปลาย, รัตนโกสินทร์ และภาพอื่น ๆ

คำสำคัญ: ประติมากรรม, ลวดลายปูนปั้น, การเรียนรู้เชิงลึก, การจำแนก, โครงข่ายประสาทเทียม
Keyword: sculpture, stucco pattern, deep learning, classification, neural network

กิตติกรรมประกาศ

โครงงานปริญญานิพนธ์นี้เป็นการสร้างเว็บแอปพลิเคชันการจำแนกยุคสมัยประติมากรรมลวดลายปูนปั้นประดับโบราณสถานไทยด้วยการเรียนรู้เชิงลึก ซึ่งสำเร็จลุล่วงไปได้ด้วยดี เนื่องจากได้รับการสนับสนุนทางด้านความรู้จากบุคคลที่ผู้วิจัยจะกล่าวถึงดังต่อไปนี้

ขอขอบพระคุณ ผู้ช่วยศาสตราจารย์โอภาส วงษ์ทวีทรัพย์อาจารย์ ที่ปรึกษาโครงงาน อาจารย์ ดร.ณัฐพงศ์ แตงจุ้ย อาจารย์ ดร.ปัญญนัท อ้นพงษ์ อาจารย์ ดร.ภูริวัจน์ วรวิชัยพัฒน์ ผู้ให้คำปรึกษาให้ความรู้ และให้คำแนะนำในการศึกษาค้นคว้าตั้งแต่เริ่มต้นจนจบปริญญานิพนธ์ฉบับนี้สำเร็จผลได้ด้วยดีและขอพระขอบคุณ

สุดท้ายนี้ขอกราบขอบพระคุณบิดา มารดา และครอบครัวที่ให้กำลังใจในทุกเรื่อง ขอบคุณเพื่อนทุกคนที่มีส่วนเกี่ยวข้องในการทำโครงงาน และขอบคุณผู้จัดทำโครงงานสำหรับความตั้งใจและความพยายามที่ทำให้โครงงานสมบูรณ์และสำเร็จตามที่ได้คาดหวังไว้รวมถึงทุกคนที่มีสวนร่วมในการช่วยเหลือให้คำแนะนำแก่ผู้วิจัย จึงขอกราบขอบพระคุณเป็นอย่างสูง ณ โอกาสนี้

ณภัทร ปัทมโรจน์

สารบัญ

หน้า

บทคัดย่อ ..... ก
กิตติกรรมประกาศ ..... ข
สารบัญ ..... ค
สารบัญตาราง ..... ฉ
สารบัญรูป ..... ช
บทที่ 1 บทนำ 1
1.1 ที่มาและความสำคัญของปัญหา 1
1.2 วัตถุประสงค์ 1
1.3 ลักษณะและขอบเขต 1
1.3.1 ส่วนการทดลอง 2
1.3.2 ส่วนการรับข้อมูล 2
1.3.3 ส่วนของการทำนาย 2
1.3.4 ส่วนการแสดงผล 2
1.4 อุปกรณ์และเครื่องมือที่ใช้ 2
1.4.1 ฮาร์ดแวร์ 2
1.4.2 ซอฟต์แวร์ 2
1.5 ผลที่คาดว่าจะได้รับ 3
1) ได้รับความรู้ในการประมวลผลภาพเพื่อประยุกต์ใช้กับงานประติมากรรมลวดลายปูนปั้น 3
2) ได้รับความรู้เกี่ยวกับปัญญาประดิษฐ์และการรู้จำวัตถุ 3
3) ได้ความรู้เรื่องลวดลายปูนปั้นในยุคสมัยต่าง ๆ 3
บทที่ 2 ผลงานที่เกี่ยวข้อง 4
2.1 Plantsnap 4
2.2 Pum-Riang Thai Silk Pattern Classification using Texture Analysis 5
2.3 CLASSIFICATION OF BUDDHA AMULETS USING DEEP CONVOLUTIONAL NEURAL NETWORKS 6
2.4 ความแตกต่างระหว่างโครงงานกับผลงานที่เกี่ยวข้อง 7
บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง 8
3.1 ลักษณะของลวดลายปูนปั้นแต่ละยุคสมัย 8
3.2 Convolutional Neural Network (CNN) 11
3.2.1 Convolutional Neural Network (CNN) 11
3.2.2 Feature Extraction 11
3.3 ResNet (Deep Residual Network) 13
3.3.1 Deep Residual Network 13
3.3.2 Vanishing Gradient 13

3.3.3 ResNet Architecture 14
3.4 Data Augmentation 15
3.4.1 เหตุใดจึงควรใช้ Data Augmentation? 15
3.4.2 เทคนิคที่ใช้ในการ Augmentation 16
3.5 Digital Image Processing 18
3.5.1 ภาพดิจิทัลคืออะไร 18
3.5.2 การแบ่งประเภทของภาพดิจิทัล 18
3.5.3 ขั้นตอนในการประมวลผลภาพ 19
บทที่ 4 ขั้นตอนและแผนการดำเนินงาน 22
4.1 ขั้นตอนการดำเนินงาน 22
4.1.1 กำหนดหัวข้อโครงงาน 22
4.1.2 กำหนดขอบเขตและวางแผนกำเนินงาน 22
4.1.3 ศึกษาความรู้และทฤษฎีที่เกี่ยวข้อง 22
4.1.4 ออกแบบระบบ 22
4.1.5 พัฒนาระบบ 22
4.1.6 ทดสอบและปรับปรุง 22
4.1.7 จัดทำเอกสารประกอบโครงงาน 22
4.2 แผนการดำเนินงาน 23
บทที่ 5 การวิเคราะห์และออกแบบระบบ 24
5.1 โครงสร้างของระบบ 24
5.1.1 ส่วนการทำงานของเว็บแอปพลิเคชัน 24
5.1.2 หน้าส่วนของผู้ใช้งาน 25
5.1.3 ที่มาของ Dataset ภาพสี 26
5.1.4 การ Augmentation 26
5.1.5 ที่มาของ Dataset ขาวเทา 27
5.2 กระบวนการทำงานทดลองและออกแบบโมเดล 27
5.2.1 การออกแบบโมเดล 27
5.2.2 ภาพสี 29
5.2.1 ภาพขาวเทา 32
5.3 สรุปผลจากการฝึกฝนโมเดล CNN จากกราฟ 34
5.4 ตารางสรุปผล 35
บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ 36
6.1 ผลการดำเนินงาน 36
6.1.1 ระบบในเว็บแอพลิเคชั่น 36
6.1.2 การทดลองโมเดล 37
6.2 ความพึงพอใจผู้ใช้งาน 42
6.2.1 ด้านความสะดวกในการใช้ 42

6.2.2 ด้านการออกแบบหน้าเว็บ 43
6.2.3 สรุปความพึงพอใจผู้ใช้ 44
6.3 ข้อจำกัด 44
6.4 ข้อเสนอแนะ 44
บรรณานุกรม 46
ภาคผนวก ก คู่มือการติดตั้งโปรแกรม 47
ก.1 วิธีการติดตั้ง Visual studio code 47
ก.1.1 ดาวน์โหลด Visual studio code ที่เว็บไซต์ code.visualstudio.com/download 47
ก.1.2 ติดตั้ง Extension Python ใน Visual Studio Code 47
ก.1.3 ติดตั้ง Extension Flask Snippet ใน Visual Studio Code 47
ก.2 การติดตั้ง Library ที่จำเป็นต่อ Project 48
ก.2.1 การติดตั้ง TensorFlow พิมพ์ pip install tensorflow ลงใน cmd 48
ก.2.2 การติดตั้ง Pillow พิมพ์ pip install Pillow ลงใน cmd 48
ก.2.3 วิธีการติดตั้ง numpy พิมพ์ pip install numpy ลงใน cmd 48
ก.2.4 วิธีการติดตั้ง Flask พิมพ์ pip install flask ลงใน cmd 48
ก.3 การติดตั้งโปรเจกต์ 49
ภาคผนวก ข การทำงานของโปรแกรม 51
ข.1 แผนผังการทำงานของระบบ 51
ข.2 ฟังก์ชันในโปรแกรม 51
ข.2.1 ฟังก์ชันปรับขนาดภาพที่รับเข้ามา 51
ข.2.2 ฟังก์ชันทำนาย 52
ข.2.3 ฟังก์ชันตรวจสอบไฟล์ และผลลัพธ์ 52
ภาคผนวก ค คู่มือการใช้โปรแกรม 53
ค.1 หน้าหลัก 53
ค.2 ขั้นตอนการใช้งาน 53
ค.2.1 การอัปโหลดภาพ 53
ค.2.2 การทำนายภาพ 55
ค.2.3 การแสดงข้อมูลเพิ่มเติม 56
ภาคผนวก ง แบบสอบถามความพึงพอใจ 57
ง.1 แบบสอบถามความพึงพอใจ 57

สารบัญตาราง

หน้า

ตารางที่ 2.1 ความแตกต่างระหว่างโครงงานกับผลงานที่เกี่ยวข้อง 7
ตารางที่ 3.1 สรุปลวดลายแต่ละยุคสมัย 10
ตารางที่ 4.1 แผนและขั้นตอนการดำเนินงาน 23
ตารางที่ 5.1 สรุปผลการทดลอง 35
ตารางที่ 6.1 ผลการทดลองโมเดล CNN kernel 5x5 ภาพสี 38
ตารางที่ 6.2 ผลการทดลองโมเดล CNN kernel 5x5 ภาพขาวเทา 38
ตารางที่ 6.3 ผลการทดลองโมเดล CNN kernel 11x11 ภาพสี 39
ตารางที่ 6.4 ผลการทดลองโมเดล CNN kernel 11x11 ภาพขาวเทา 39
ตารางที่ 6.5 ผลการทดลองโมเดล CNN kernel 15x15 ภาพสี 39
ตารางที่ 6.6 ผลการทดลองโมเดล CNN kernel 15x15 ภาพขาวเทา 40
ตารางที่ 6.7 ผลการทดลองโมเดล ResNet50 ภาพสี 40
ตารางที่ 6.8 ผลการทดลองโมเดล ResNet50 ภาพขาวเทา 40
ตารางที่ 6.9 สรุปความแม่นยำสูงสุดแต่ละคลาสของโมเดลภาพสี 41
ตารางที่ 6.10 สรุปความแม่นยำสูงสุดแต่ละคลาสของโมเดลภาพสี 41
ตารางที่ 6.11 สรุปความพึงพอใจผู้ใช้ 44

สารบัญรูป

หน้า

รูปที่ 2.1 Plantsnap 4
รูปที่ 2.2 งานวิจัยการรู้จำลวดลายของผ้าไหมพุมเรียง 5
รูปที่ 2.3 งานวิจัยการจำแนกพระเครื่องด้วยการเรียนรู้เชิงลึก 6
รูปที่ 3.1 ลวดลวดลายปูนปั้นสมัยสุโขทัย 8
รูปที่ 3.2 ลวดลวดลายปูนปั้นสมัยอยุธยาตอนต้น 8
รูปที่ 3.3 ลวดลวดลายปูนปั้นสมัยอยุธยาตอนกลาง 9
รูปที่ 3.4 ลวดลวดลายปูนปั้นสมัยอยุธยาตอนปลาย 9
รูปที่ 3.5 ลวดลวดลายปูนปั้นสมัยรัตนโกสินทร์ 10
รูปที่ 3.6 Filter 11
รูปที่ 3.7 Stride และ Padding 12
รูปที่ 3.8 Max pooling 12
รูปที่ 3.9 shortcut connection 13
รูปที่ 3.10 Bottleneck Building Block 14
รูปที่ 3.11 ResNet Block 14
รูปที่ 3.12 Bottleneck block และ Normal block 15
รูปที่ 3.13 ตัวอย่างโค้ดการ Rotation 16
รูปที่ 3.14 ตัวอย่างโค้ดการ Translation 16
รูปที่ 3.15 ตัวอย่างโค้ดการ Scailing 16
รูปที่ 3.16 ตัวอย่างโค้ดการ Flipping 16
รูปที่ 3.17 ตัวอย่างโค้ดการ Shearing 17
รูปที่ 3.18 ตัวอย่างโค้ดการ Zooming 17
รูปที่ 3.19 ตัวอย่างโค้ดการ Brightness Adjustment 17
รูปที่ 3.20 ตัวอย่างโค้ดการ Contrast Adjustment 17
รูปที่ 3.21 ตัวอย่างโค้ดการทำ Noise Addition 18
รูปที่ 3.22 Binary Image, Grayscale Image, RGB Image and Indexed Image 18
รูปที่ 3.23 ตัวอย่างโค้ดการนำภาพเข้าระบบ 19
รูปที่ 3.24 ตัวอย่างโค้ดการปรับคอนทราสต์ 19
รูปที่ 3.25 ตัวอย่างโค้ดลดสัญญาณรบกวน 19
รูปที่ 3.26 ตัวอย่างโค้ดการแยกแชแนลสี R G B 19
รูปที่ 3.27 ตัวอย่างโค้ดการใช้เทคนิค Wavelet 20
รูปที่ 3.28 ตัวอย่างโค้ดการบีบอัด JPEG 20
รูปที่ 3.29 ตัวอย่างโค้ดการลบจุดสีขาวในภาพไบนารี 20
รูปที่ 3.30 ตัวอย่างโค้ดแบ่งส่วนของภาพด้วย thresholding เพื่อตัดภาพเป็นขาว-ดำ 20

รูปที่ 3.31 ตัวอย่างโค้ดการค้นหาและวาดขอบเขตของวัตถุที่อยู่ในภาพ 21
รูปที่ 3.32 ตรวจจับใบหน้าในภาพโดยใช้ Haar Cascade 21
รูปที่ 5.1 โฟลวชาร์ตการทำงานของเว็บแอปพลิเคชัน 24
รูปที่ 5.2 หน้าส่วนของผู้ใช้งาน 25
รูปที่ 5.3 โค้ดที่ใช้ในการทำ Augmentation 26
รูปที่ 5.4 ตัวอย่างก่อนและหลังทำการ Augmentation 27
รูปที่ 5.5 โค้ดที่ใช้ในการแปลงภาพสีเป็นขาวเทา 27
รูปที่ 5.6 โมเดล CNN ในการทดลอง 28
รูปที่ 5.7 จำนวน epoch และการ fit 28
รูปที่ 5.8 โครงสร้างของโมเดล 28
รูปที่ 5.9 ความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล CNN kernel 5x5 ภาพสี 29
รูปที่ 5.10 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล CNN kernel 11x11 ภาพสี 30
รูปที่ 5.11 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล CNN kernel 15x15 ภาพสี 30
รูปที่ 5.12 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล ResNet50 ภาพสี 31
รูปที่ 5.13 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล CNN kernel 5x5 ภาพขาวเทา 32
รูปที่ 5.14 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล CNN kernel 11x11 ภาพขาวเทา 33
รูปที่ 5.15 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล CNN kernel 15x15 ภาพขาวเทา 33
รูปที่ 5.16 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล ResNet50 ภาพขาวเทา 34
รูปที่ 6.1 ระบบเลือกไฟล์ภาพ 36
รูปที่ 6.2 ระบบแสดงภาพที่ผู้ใช้อัปโหลด 36
รูปที่ 6.3 ระบบหมุนภาพที่ผู้ใช้อัปโหลด 37
รูปที่ 6.4 ระบบแสดงความมั่นใจของโมเดลต่อภาพที่ทำนาย 37
รูปที่ 6.5 ผลการประเมินความง่ายในการใช้งาน 42
รูปที่ 6.6 ผลการประเมินประสิทธิภาพการใช้งาน 42
รูปที่ 6.7 ผลการประเมินฟังก์ชันการใช้งาน 43
รูปที่ 6.8 ผลการประเมินความสวยงามหน้า UI 43
รูปที่ 6.9 ผลการประเมินความเข้าใจง่ายของหน้า UI 43

รูปผนวกที่ ก.1 เว็บไซต์ code.visualstudio.com/download 47

รูปผนวกที่ ก.2 Extension Python ใน Visual Studio Code 47
รูปผนวกที่ ก.3 Extension Flask Snippet ใน Visual Studio Code 47
รูปผนวกที่ ก.4 pip install tensorflow ใน cmd 48
รูปผนวกที่ ก.5 pip install Pillow ใน cmd 48
รูปผนวกที่ ก.6 pip install numpy ใน cmd 48
รูปผนวกที่ ก.7 pip install flask ใน cmd 48
รูปผนวกที่ ก.8 หน้าเว็บ Github สำหรับดาวน์โหลดไฟล์โปรเจกต์ 49
รูปผนวกที่ ก.9 ไฟล์โปรเจกต์ภายในโฟล์เดอร์ 49
รูปผนวกที่ ก.10 รัน Project ที่ app.py 50

รูปผนวกที่ ข.1 แผนผังการทำงานของระบบ 51
รูปผนวกที่ ข.2 ฟังก์ชันปรับขนาดภาพที่รับเข้ามา 51
รูปผนวกที่ ข.3 ฟังก์ชันทำนาย 52
รูปผนวกที่ ข.4 ฟังก์ชันตรวจสอบไฟล์ และผลลัพธ์ 52

รูปผนวกที่ ค.1 หน้าหลัก 53
รูปผนวกที่ ค.2 หน้าต่างเลือกภาพ 53
รูปผนวกที่ ค.3 หน้าแสดงภาพที่ผู้ใช้เลือกมา 54
รูปผนวกที่ ค.4 ดรอปดาวน์การหมุน 54
รูปผนวกที่ ค.5 ระหว่างรอทำนายผล 55
รูปผนวกที่ ค.6 หน้าแสดงผลทำนาย 55
รูปผนวกที่ ค.7 ลิงก์ "เพิ่มเติม" 56
รูปผนวกที่ ค.8 ลิงก์ "ลดลง" 56

รูปผนวกที่ ง.1 แบบสอบถามความพึงพอใจ 57

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของปัญหา

ประติมากรรมตามพจนานุกรมราชบัณฑิตยสถานมีความหมายว่า ศิลปะสาขาหนึ่งในจำพวกวิจิตรศิลป์เกี่ยวกับการแกะสลักไม้ หินอ่อน โลหะ ให้เป็นรูปหรือลวดลายต่าง ๆ ในประเทศไทยนิยมนำปูนมาปั้นเป็นลวดลายเพื่อประดับศาสนสถานหรือโบราณสถานต่าง ๆ

ลวดลายปูนปั้นในโบราณสถานของไทยเป็นสมบัติทางวัฒนธรรมอันล้ำค่าที่สะท้อนให้เห็นถึงประวัติศาสตร์ ศิลปะและวัฒนธรรมของชาติไทยมาช้านาน ประติมากรรมลวดลายปูนปั้นโบราณเหล่านี้มีอายุตั้งแต่สมัยทวารวดีจนถึงสมัยรัตนโกสินทร์แต่ละยุคสมัยมีรูปแบบและลักษณะเฉพาะตัวที่บ่งบอกถึงพัฒนาการทางศิลปะและวัฒนธรรมในช่วงเวลานั้น ๆ

การระบุยุคสมัยของประติมากรรมลวดลายปูนปั้นโบราณเป็นขั้นตอนสำคัญในการศึกษาการระบุยุคสมัยแบบดั้งเดิมอาศัยการสังเกตลักษณะทางกายภาพและรูปแบบของงานศิลปะเป็นหลักซึ่งอาจทำได้ยากและใช้เวลานานโดยเฉพาะกับประติมากรรมลวดลายปูนปั้นที่มีขนาดใหญ่หรือซับซ้อน นอกจากนี้การสังเกตลักษณะทางกายภาพและรูปแบบของงานศิลปะอาจมีความคลาดเคลื่อนได้ เนื่องจากอาจได้รับผลกระทบจากปัจจัยต่าง ๆ เช่น สภาพแวดล้อม สภาพการใช้งาน เป็นต้น

ในปัจจุบันมีการปัญญาประดิษฐ์และการเรียนรู้ด้วยเครื่องและการเรียนรู้เชิงลึกเพื่อทำการรู้จำวัตถุในภาพ งานวิจัยนี้ก็ได้มุ่งหวังที่จะนำมาประยุกต์ใช้เพื่อจำแนกยุคสมัยของลวดลายปูนปั้นได้ เนื่องจากหากเป็นบุคคลธรรมอาจจะไม่สามารถแยกยุคสมัยของประติมากรรมปูนปั้นได้อย่างรวดเร็ว

1.2 วัตถุประสงค์

1) เพื่อศึกษาวิธีการและขั้นตอนการประมวลผลภาพสำหรับประติมากรรมลวดลายปูนปั้น
2) เพื่อศึกษาทดลองหาโครงสร้างของนิวรอนเน็ตเวิร์คที่เหมาะสมในการจำแนกยุคสมัยของลวดลายปูนปั้น
3) เพื่อเป็นส่วนหนึ่งในการสนับสนุนการศึกษาสมบัติทางวัฒนธรรมอันล้ำค่าของประเทศไทย

1.3 ลักษณะและขอบเขต

เป็นเว็บแอปพลิเคชันในแบบไดนามิค มีการจำแนกยุคสมัยของภาพปูนปั้น โดยใช้โครงข่ายประสาทเทียม

ข้อมูลภาพ dataset ที่ได้มาจากการลงพื้นที่ มี 1757 ภาพ

โดยแบ่งเป็น

1. สุโขทัย 176 ภาพ
2. อยุธยาตอนต้น 444 ภาพ
3. อยุธยาตอนกลาง 203 ภาพ
4. อยุธยาตอนปลาย 584 ภาพ
5. รัตนโกสินทร์ 350 ภาพ

1.3.1 ส่วนการทดลอง

1. ทำการ Resize ภาพทั้งหมดเป็น 224x224 pixel และทำการ Augment
2. ในการทดลองนี้ มีการใช้โมเดลสองแบบ ได้แก่ CNN และ ResNet50 สำหรับโมเดล CNN นั้นจะถูกนำมาทดลองในสองรูปแบบของข้อมูลภาพ คือ ภาพสี และ ภาพขาวเทา และจะทดลองในขนาด kernel ที่ต่างกัน 3 ขนาดคือ 5x5, 11x11 และ 15x15
3. ส่วนของโมเดล ResNet50 นั้นจะนำมาทดลองกับ ภาพสี และ ภาพขาวเทา เช่นเดียวกับ CNN เพื่อให้สามารถเปรียบเทียบผลลัพธ์จากสองโมเดลได้โดยตรง

1.3.2 ส่วนการรับข้อมูล

1. ผู้ใช้เลือกไฟล์ภาพขนาดขั้นต่ำ 224x224 พิกเซล สามารถเพิ่มได้ในช่องรับรูปภาพครั้งละ 1 รูปต่อการกดทำนาย 1 ครั้ง
2. ผู้ใช้สามารถเลือกไฟล์ภาพใหม่ได้ โดยการกดเลือกไฟล์ภาพอีกครั้ง
3. ผู้ใช้สามารถกดหมุนภาพได้

1.3.3 ส่วนของการทำนาย

1. ทางเว็บแอพพลิเคชั่นจะนำภาพที่รับมาจากผู้ใช้ เข้าสู่กระบวนการเตรียมภาพ ให้ภาพเป็นขนาด 224x224 พิกเซล
2. นำภาพที่ปรับขนาดแล้วส่งไปยังโมเดลเพื่อทำการทำนายผล
3. ส่งผลลัพธ์ความน่าจะเป็นไปยังส่วนแสดงผล

1.3.4 ส่วนการแสดงผล

1. แสดงผลลัพธ์ที่มีความมั่นใจสูงที่สุด ออกมาในรูปแบบข้อความ พร้อมเปอร์เซ็นต์ความมั่นใจ
2. แสดงความน่าจะเป็นของแต่ละคลาสเป็นหลอดสีที่แสดงความมั่นใจของคลาสนั้นพร้อมทั้งแสดงเลขเปอร์เซ็นต์
3. ในส่วนของหน้าเว็บแอพพลิเคชั่นมีการแสดงข้อความที่กล่าวถึงโครงงาน และลวดลายปูนปั้น

1.4 อุปกรณ์และเครื่องมือที่ใช้

1.4.1 ฮาร์ดแวร์

- หน่วยประมวลผลกลาง 13th gen intel core i5-13400f
- หน่วยความจำชั่วคราว ขนาด 16 GB
- ฮาร์ดดิสก์ KC3000 PCIe 4.0 NVMe M.2 SSD ขนาด 512 GB
- หน่วยประมวลผลกราฟฟิก NVIDIA® GeForce® RTX 4060TI ขนาด 8 GB

1.4.2 ซอฟต์แวร์

- ระบบปฏิบัติการ Window 10 Home Single Language
- ภาษาที่ใช้พัฒนาโมเดลปัญญาประดิษฐ์ Python เวอร์ชั่น 3.11.5

- โปรแกรมสำหรับปรับแต่งโค้ด Pycharm เวอร์ชั่น 2023.3
- โปรแกรมสำหรับปรับแต่งโค้ด Vscode เวอร์ชั่น 1.85
- โปรแกรมสำหรับพัฒนาเว็บแอพลิเคชั่น flask เวอร์ชั่น 3.0.0
- ผู้ให้บริการทำสร้างโมเดลในการพัฒนา Google Colab
- เว็ปบราวเซอร์ที่ใช้ทดสอบ Google Chrome เวอร์ชั่น 120.0.6099.129/130

1.5 ผลที่คาดว่าจะได้รับ

1) ได้รับความรู้ในการประมวลผลภาพเพื่อประยุกต์ใช้กับงานประติมากรรมลวดลายปูนปั้น
2) ได้รับความรู้เกี่ยวกับปัญญาประดิษฐ์และการรู้จำวัตถุ
3) ได้ความรู้เรื่องลวดลายปูนปั้นในยุคสมัยต่าง ๆ

บทที่ 2 ผลงานที่เกี่ยวข้อง

เป็นบทที่กล่าวถึงผลงานที่เกี่ยวข้อง กับการทำโครงงานที่มีลักษณะคล้ายคลึงกับโครงงานที่จะดำเนินการเพื่อนำมาใช้เป็นข้อมูลอ้างอิง ข้อดี และข้อเสียหรือข้อจำกัดของผลงานที่เกี่ยวข้องเพื่อแสดงให้เห็นถึงประเด็นปัญหาและรายละเอียดแนวคิดเพื่อนำมาศึกษา ดังนี้

2.1 Plantsnap

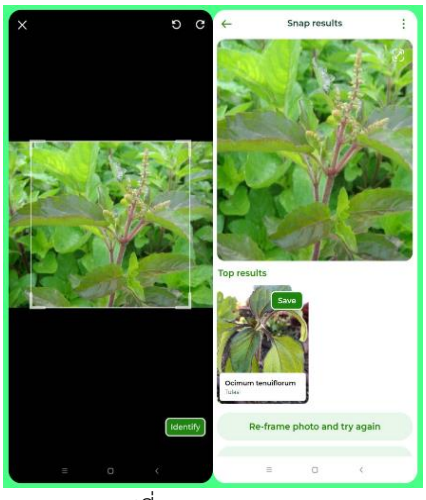
Plantsnap [1] เป็นแอปพลิเคชันที่ออกแบบมาเพื่อช่วยผู้ใช้ระบุพืชหรือต้นไม้ต่าง ๆ ด้วยการใช้ภาพถ่ายที่ถ่ายจากโทรศัพท์มือถือหรืออุปกรณ์เครื่องถ่ายภาพอื่น ๆ Plantsnap จะวิเคราะห์และรับรู้ชนิดของพืชและแสดงชื่อของพืชดังกล่าวขึ้นบนหน้าจอทันที

การใช้งาน Plantsnap มีขั้นตอนง่าย ๆ ดังนี้:

ถ่ายภาพ: ใช้กล้องของโทรศัพท์มือถือหรืออุปกรณ์ถ่ายภาพอื่น ๆ เพื่อถ่ายภาพของพืชที่คุณต้องการระบุ

อัปโหลด: นำภาพที่ถ่ายมาอัปโหลดลงในแอป Plantsnap.

ผลลัพธ์: แอปจะประมวลผลภาพและให้ข้อมูลเกี่ยวกับชนิดของพืชนั้น ๆ รวมถึงข้อมูลทางวิทยาศาสตร์ เช่น ชื่อทางวิทยาศาสตร์และลักษณะทางกายภาพของพืช



รูปที่ 2.1 Plantsnap

ข้อดีของ Plantsnap:

1. ความสะดวกสบาย การใช้งานง่าย ๆ และไม่ต้องการความเชี่ยวชาญในการระบุพืช
2. มีฐานข้อมูลที่ใหญ่ขึ้นทุกวันซึ่งช่วยให้มีความแม่นยำในการระบุพืช
3. ผู้ใช้สามารถแชร์รูปภาพของพืชที่พวกเขาถ่ายได้กับชุมชน Plantsnap เพื่อขอความช่วยเหลือในการระบุ
4. การให้ข้อมูลเกี่ยวกับพืชและต้นไม้ที่ถูกต้องช่วยให้ผู้ใช้เรียนรู้เพิ่มเติมเกี่ยวกับธรรมชาติและสิ่งแวดล้อม

ข้อเสียของ Plantsnap:

1. บางครั้งการระบุพืชอาจไม่แม่นยำ 100% โดยเฉพาะเมื่อเปรียบเทียบกับฐานข้อมูลที่ใหญ่ของพืชต่าง ๆ
2. บางเวอร์ชั่นของแอปอาจมีค่าใช้จ่าย ทำให้ไม่สะดวกสำหรับบางผู้ใช้

3. บางครั้งอาจมีข้อจำกัดในการระบุพืชบางชนิดที่มีลักษณะทางกายภาพที่คล้ายกันหรือมีการปรับแต่งข้อมูลที่ไม่สมบูรณ์ในฐานข้อมูล

2.2 Pum-Riang Thai Silk Pattern Classification using Texture Analysis

งานวิจัยนี้เป็นงานวิจัยที่ใช้ในการรู้จำลวดลายของผ้าไหมพุมเรียง [2]

เทคโนโลยีหลักที่งานวิจัยนี้ ใช้:

image enhancement:

- ใช้ปรับปรุงรูปลักษณ์หรือคุณภาพของภาพ เป้าหมายคือทำให้ภาพเหมาะสมยิ่งขึ้นสำหรับการใช้งานเฉพาะ

Feature Extraction:

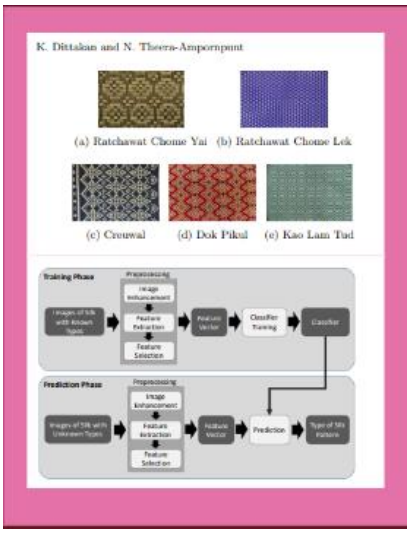
- ใช้แปลงข้อมูลดิบให้เป็นชุดของคุณลักษณะที่สามารถใช้ได้อย่างมีประสิทธิภาพสำหรับงานต่าง ๆ เช่น การจำแนกประเภท, การตรวจจับ และการรับรู้

Feature Selection:

- ใช้เลือกคุณลักษณะที่เกี่ยวข้องสำหรับการสร้างโมเดล เป้าหมายคือปรับปรุงประสิทธิภาพของโมเดลโดยการลบคุณลักษณะที่ซ้ำซ้อน, ไม่เกี่ยวข้อง, หรือมีเสียงรบกวน

Deep Learning:

- ใช้ในการพัฒนา AI ที่มีความซับซ้อนมากขึ้น
-



รูปที่ 2.2 งานวิจัยการรู้จำลวดลายของผ้าไหมพุมเรียง

ข้อดีของงานวิจัยนี้:

1. สามารถช่วยในการศึกษาลวดลายผ้าไหมไทย ซึ่งมีความสำคัญทางวัฒนธรรมและประวัติศาสตร์ โดยใช้เทคโนโลยีในการวิเคราะห์ลวดลายอย่างแม่นยำ
2. นำเทคโนโลยีการวิเคราะห์ภาพและการประมวลผลภาพมาช่วยในการจำแนกลวดลาย ซึ่งสามารถประยุกต์ใช้ในระบบอัตโนมัติหรือฐานข้อมูลผ้าไหม
3. ช่วยให้การจำแนกลวดลายผ้าไหมในอุตสาหกรรมเป็นไปอย่างรวดเร็วและมีประสิทธิภาพมากขึ้น

ข้อเสียของงานวิจัยนี้:

1. อาจมีข้อจำกัดในการวิเคราะห์ลวดลายที่มีความซับซ้อนหรือมีลักษณะคล้ายคลึงกัน ซึ่งอาจทำให้ระบบมีข้อผิดพลาดในการจำแนก
2. อาจเป็นปัญหาหากไม่มีฐานข้อมูลที่ครอบคลุม
3. การการเปลี่ยนแปลงในลักษณะของผ้าไหม เช่น สีหรือความละเอียด อาจส่งผลต่อความแม่นยำของการวิเคราะห์และการจำ

2.3 CLASSIFICATION OF BUDDHA AMULETS USING DEEP CONVOLUTIONAL NEURAL NETWORKS

งานวิจัยนี้เป็นงานวิจัยที่ใช้ในการรู้จำพระเครื่อง และเหรียญโลหะ [3]

เทคโนโลยีหลักที่งานวิจัยนี้ ใช้:

Image Segmentation algorithm:

- ใช้ตรวจสอบหาพื้นที่ที่วัตถุนั้นอยู่เพื่อระบุว่าเป็นวัตถุชนิดใด โดยการพยายามตีกรอบวัตถุ ต่าง ๆ ที่อยู่ในรูปภาพ

Image Preprocessing:

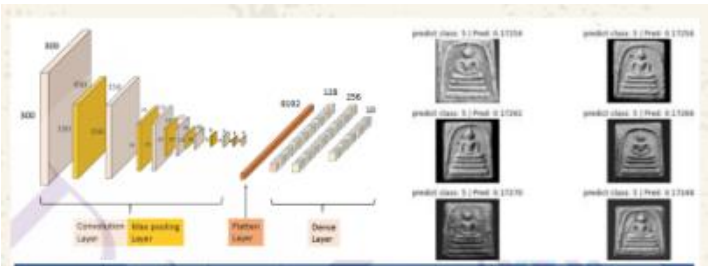
- ใช้เตรียมข้อมูลภาพก่อนนำภาพไปใช้ในโมเดล CNN การปรับขนาดภาพ การลบสัญญาณรบกวน หรือการปรับความคมชัด

Pooling Layers:

- ใช้เพื่อลดมิติของข้อมูลและป้องกันการเกิด overfitting ขณะเดียวกันยังช่วยคงคุณสมบัติสำคัญของภาพเอาไว้

Deep Learning model:

- ใช้โมเดลต่าง ๆ ในการพัฒนา แยกแต่ละรูปแบบเพื่อเพิ่มประสิทธิภาพ
-



Model	Parameter	Epoch	Learning rate	Loss functions	Optimizer	Transfer learning/ Train from scratch
Custom CNN	2,662,122	100	0.001	SparseCategoricalCrossentropy	Adam	None
Custom VGG16	9,291,850	100	0.001	SparseCategoricalCrossentropy	Adam	None
ResNet50	23,564,800	100	0.001	CategoricalCrossentropy	Adam	imagenet
EfficientNetB0	5,730,169	100	0.001	CategoricalCrossentropy	Adam	imagenet
EfficientNetB3	10,783,535	100	0.001	CategoricalCrossentropy	Adam	imagenet
Siamese	13,875,668	100	0.001	Margin	RMSprop	None
Custom Xception	4,220,906	500	0.001	CategoricalCrossentropy	Adam	None

รูปที่ 2.3 งานวิจัยการจำแนกพระเครื่องด้วยการเรียนรู้เชิงลึก

ข้อดีของงานวิจัยนี้:

1. สามารถช่วยในการศึกษาพระเครื่อง ซึ่งมีความสำคัญทางวัฒนธรรมและประวัติศาสตร์ โดยใช้เทคโนโลยีในการวิเคราะห์ลวดลายอย่างแม่นยำ
2. นำเทคโนโลยีการวิเคราะห์ภาพและการประมวลผลภาพมาช่วยในการจำแนก ซึ่งสามารถประยุกต์ใช้ในระบบอัตโนมัติหรือสร้างฐานข้อมูลพระเครื่องที่ครอบคลุม
3. ช่วยให้การจำแนกพระเครื่องในเชิงพาณิชย์หรือการวิจัยเป็นไปอย่างรวดเร็วและมีประสิทธิภาพมากขึ้น

ข้อเสียของงานวิจัยนี้:

1. อาจมีข้อจำกัดในการวิเคราะห์ลวดลายที่มีความซับซ้อนหรือมีลักษณะคล้ายคลึงกัน ซึ่งอาจทำให้ระบบมีข้อผิดพลาดในการจำแนก
2. อาจเป็นปัญหาหากไม่มีฐานข้อมูลที่ครอบคลุม
3. การเปลี่ยนแปลงในลักษณะของพระเครื่อง เช่น การสึกหรอ หรือความแตกต่างเล็กน้อยในรายละเอียด อาจส่งผลต่อความแม่นยำของการวิเคราะห์และการจำแนก

2.4 ความแตกต่างระหว่างโครงงานกับผลงานที่เกี่ยวข้อง

ตารางนี้เปรียบเทียบคุณสมบัติของแอปพลิเคชันและโครงงาน ได้แก่ PlantSnap, Pum-Riang Silk Classification, Classification of Buddha Amulets และโครงงานของเรา

ตารางที่ 2.1 ความแตกต่างระหว่างโครงงานกับผลงานที่เกี่ยวข้อง

ข้อเปรียบเทียบ	PlantSnap	Pum-Riang silk Classification	Classification of Buddha Amulets	โครงงาน
จำแนกยุคสมัยปูนปั้นไทยอย่างเฉพาะเจาะจง	-	-	-	✓
ไม่ต้องติดตั้งลงอุปกรณ์	-	✓	✓	✓
มีหน้าเว็บแอปพลิเคชัน	✓	-	-	✓

บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง

ในบทนี้จะกล่าวถึง ส่วนของทฤษฎีและความรู้ที่เกี่ยวข้อง ที่นำมาพัฒนาและออกแบบระบบ เพื่อเป็นแนวทางการพัฒนาระบบโครงงานวิจัยนี้โดยมีรายละเอียดดังนี้

3.1 ลักษณะของลวดลายปูนปั้นแต่ละยุคสมัย

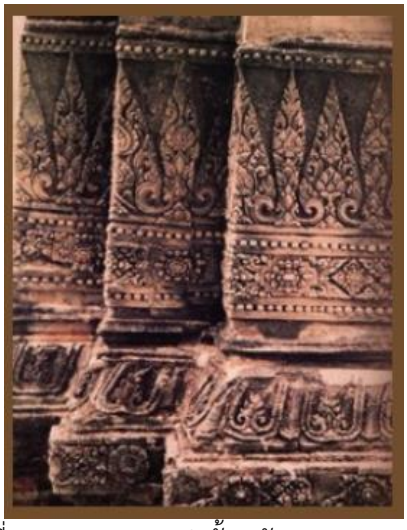
- **ยุคสมัยสุโขทัย** [4]**:**
    - ลวดลวดลายปูนปั้นส่วนใหญ่มีอิทธิพลจากขอมโบราณ และจีน
    - ลวดลายนิยมใช้ มี ลายพรรณพฤกษา ลายดอกโบตั๋น ลายหรูอี้ ลายรักร้อยแข้งสิงห์ ดังรูปที่ 3.1
    - เน้นความเรียบง่าย สื่อถึงความศรัทธา

รูปที่ 3.1 ลวดลวดลายปูนปั้นสมัยสุโขทัย

- **ยุคสมัยอยุธยาตอนต้น** [5]**:**
    - ลวดลายปูนปั้นได้รับอิทธิพลจากสุโขทัย
    - ลวดลายนิยมใช้ มี ลายกรุยเชิง ลายกนก ลายพรรณพฤกษา ลายเฟื่องอุบะ ลายดอกไม้ ลายกลีบบัว ดังรูปที่ 3.2
    - เน้นความประณีต อ่อนช้อย

รูปที่ 3.2 ลวดลวดลายปูนปั้นสมัยอยุธยาตอนต้น

- **ยุคสมัยอยุธยาตอนกลาง** [6]**:**
    - ลวดลายปูนปั้นได้รับอิทธิพลจากจีน ล้านนา และอยุธยาตอนต้น
    - ลวดลายนิยมใช้ มี ลายกรุยเชิง ลายหน้ากระดาน ลายกลีบบัว ลายดอกกลม ลายดอกจอก ดังรูปที่ 3.3
    - เน้นความยิ่งใหญ่ อลังการ



รูปที่ 3.3 ลวดลวดลายปูนปั้นสมัยอยุธยาตอนกลาง

- **ยุคสมัยอยุธยาตอนปลาย** [7]**:**
    - ลวดลายปูนปั้นได้รับอิทธิพลจากตะวันตก
    - ลวดลายนิยมใช้ มี ลายประจำยาม ลายดอกจอก ลายดอกรี ลายเครือเถา ลายก้านขด ลายกลีบบัว ดังรูปที่ 3.4
    - เน้นความสมจริง สวยงาม

รูปที่ 3.4 ลวดลวดลายปูนปั้นสมัยอยุธยาตอนปลาย

- **ยุคสมัยรัตนโกสินทร์** [4]**:**
    - ลวดลายปูนปั้นมีความหลากหลาย ผสมผสานจากหลายวัฒนธรรม
    - ลวดลายนิยมใช้ มี พรรณพฤกษา ลายกระหนก ลายสิ่งมีชีวิต ลายเทพปกรณัม ดังรูปที่ 3.5
    - เน้นความวิจิตรบรรจง แสดงถึงฝีมือช่าง

รูปที่ 3.5 ลวดลวดลายปูนปั้นสมัยรัตนโกสินทร์

ตารางที่ 3.1 สรุปลวดลายแต่ละยุคสมัย

ยุคสมัย	ลวดลาย	ลักษณะ
สุโขทัย	ลายพรรณพฤกษา ลายดอกโบตั๋น ลายหรูอี้ ลายรักร้อยแข้งสิงห์	เรียบง่าย สื่อถึงความศรัทธา
อยุธยาตอนต้น	ลายกรุยเชิง ลายกนก ลายพรรณพฤกษา ลายเฟื่องอุบะ ลายดอกไม้ ลายกลีบบัว	ประณีต อ่อนช้อย
อยุธยาตอนกลาง	ลายกรุยเชิง ลายหน้ากระดาน ลายกลีบบัว ลายดอกกลม ลายดอกจอก	ยิ่งใหญ่ อลังการ
อยุธยาตอนปลาย	ลายประจำยาม ลายดอกจอก ลายดอกรี ลายเครือเถา ลายก้านขด ลายกลีบบัว	สมจริง สวยงาม
รัตนโกสินทร์	พรรณพฤกษา ลายกระหนก ลายสิ่งมีชีวิต ลายเทพปกรณัม	วิจิตรบรรจง แสดงถึงฝีมือช่าง

ดังตารางที่ 3.1 เป็นการสรุปลวดลายปูนปั้นในแต่ละยุคสมัยว่าปรากฏลวดลายใดบ้างในยุคสมัยนั้น ๆ และลักษณะโดยรวมว่าเป็นอย่างไร

3.2 Convolutional Neural Network (CNN)

3.2.1 Convolutional Neural Network (CNN) [8]

หรือ โครงข่ายประสาทแบบคอนโวลูชัน เป็นโครงข่ายประสาทเทียมหนึ่งในกลุ่ม bio-inspired โดยที่ CNN จะจำลองการมองเห็นของมนุษย์ที่มองพื้นที่เป็นที่ย่อย ๆ และนำกลุ่มของพื้นที่ย่อย ๆ มาผสานกัน เพื่อดูว่าสิ่งที่เห็นอยู่เป็นอะไรกันแน่

การมองพื้นที่ย่อยของมนุษย์จะมีการแยกคุณลักษณะ (feature) ของพื้นที่ย่อยนั้น เช่น ลายเส้นและการตัดกันของสี ซึ่งการที่มนุษย์รู้ว่าพื้นที่ตรงนี้เป็นเส้นตรงหรือสีตัดกัน เพราะมนุษย์ดูทั้งจุดที่สนใจและบริเวณรอบ ๆ ประกอบกัน

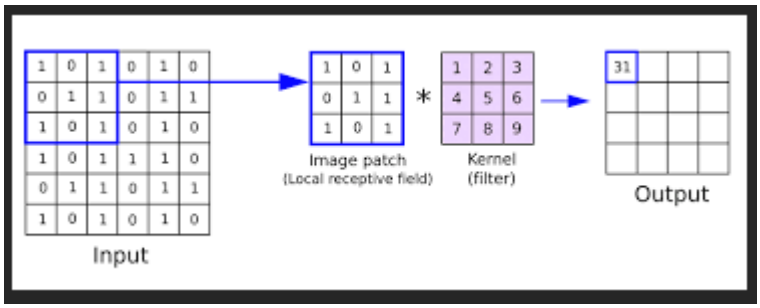
3.2.2 Feature Extraction

คือกระบวนการที่นำเข้าข้อมูลและแปลงเป็นลักษณะ (features) ที่มีความสำคัญเพื่อให้ง่ายต่อการวิเคราะห์และการประมวลผลข้อมูลต่อไป โดยลักษณะที่ถูกสกัดออกมานั้น จะเป็นข้อมูลที่สามารถแทนตัวได้เพียงพอสำหรับงานที่ต้องการ

การคำนวณเริ่มจากการกำหนดค่าใน **ตัวกรอง (filter)** หรือ **เคอร์เนล(kernel)** ที่ช่วยดึงคุณลักษณะที่ใช้ในการรู้จำวัตถุออก โดยปกติตัวกรอง/เคอร์เนลอันหนึ่ง จะดึงคุณลักษณะที่สนใจออกมาได้หนึ่งอย่าง เราจึงจำเป็นต้องตัวกรองหลายตัวกรองด้วย เพื่อหาคุณลักษณะทางพื้นที่หลาย อย่างประกอบกัน

3.2.2.1 Filter

เป็นเมทริกซ์ขนาดเล็กที่เคลื่อนที่ไปทั่วทั้งภาพนำเข้า (input image) และนำค่าของพิกเซลที่ทับกันทำการคูณกับค่าพิกเซลของภาพนำเข้าในแต่ละตำแหน่ง ผลลัพธ์จากการคูณทั้งหมดถูกรวมกันเพื่อสร้าง feature map ซึ่งเป็นผลลัพธ์หลังจากการทำ Convolutional Operation และ ในรูปที่ 3.6 แสดงถึงวิธีการดำเนินการของ Filter



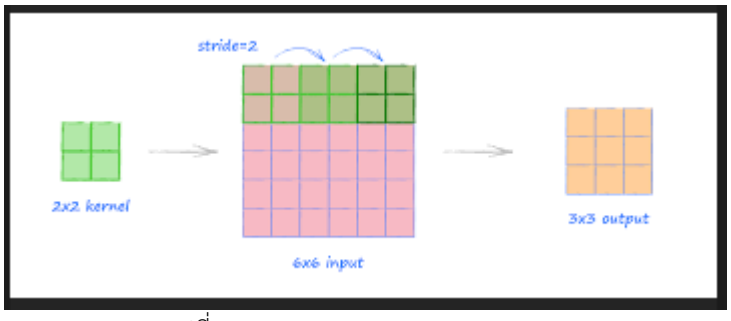
รูปที่ 3.6 Filter

3.2.2.1 Stride และ Padding

Stride เป็นตัวกำหนดว่าเราจะเลื่อนตัวกรอง (filter) ไปด้วย Step เท่าไร เราสามารถกำหนดค่าของ Stride ให้มากขึ้นก็ได้

ถ้าเราต้องการให้การคำนวนหาคุณลักษณะมีพื้นที่ทับซ้อนกันน้อยขึ้น แต่อย่างไรก็ตาม การกำหนดค่าของ Stride ที่มากขึ้นจะทำให้เราได้ผังคุณลักษณะ (feature map) ที่มีขนาดเล็กลง

Padding จากรูปด้านล่างเราจะพื้นที่สีเทารอบ ๆ Input พื้นที่เหล่านี้เป็นพื้นที่ที่เรามักเติมเข้าไป โดยอาจจะเป็นเติม 0 หรือค่าต่าง ๆ เข้าไป เพื่อให้เวลาในการทำ CNN นั้น Feature Map ที่ได้ยังคงมีขนาดเท่ากับ Input และ ในรูปที่ 3.7 แสดงถึงวิธีการดำเนินการของ Stride และ Padding

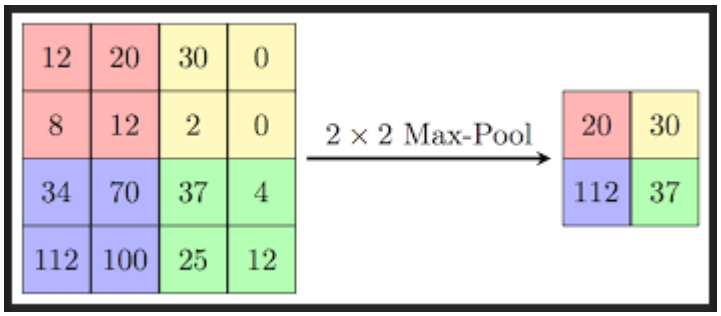


รูปที่ 3.7 Stride และ Padding

3.2.2.1 Max Pooling

Pooling คือความสามารถในการย่อรูปแบบหนึ่ง ซึ่งมีสองประเภทหลักที่นิยมกันคือ max pooling และ mean pooling

Max Pooling เป็นตัวกรองแบบหนึ่งที่หาค่าสูงสุดในบริเวณที่ตัวกรองทาบอยู่มาเป็นผลลัพธ์ โดยเราจะเตรียมตัวกรองในลักษณะเดียวกับการทำ Feature Extraction ของ CNN มาทาบบนข้อมูล แล้วเลือกค่าที่สูงที่สุดบนตัวกรองนั้นมาเป็นผลลัพธ์ใหม่ และจะเลื่อนตัวกรองไปตาม Stride ที่กำหนดไว้ โดยขนาดตัวกรองของการทำ max pooling จะนิยมเรียกกันว่า pool size และ ในรูปที่ 3.8 แสดงถึงวิธีการดำเนินการของ Max pooling



รูปที่ 3.8 Max pooling

3.3 ResNet (Deep Residual Network)

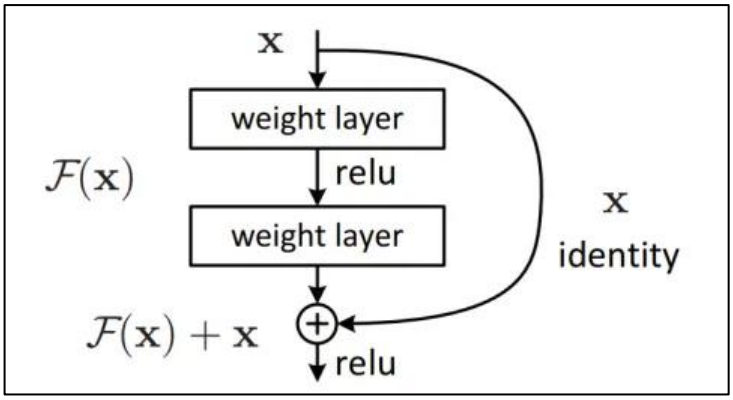
3.3.1 Deep Residual Network [9]

Deep Residual Network หรือ ResNet ถูกนำเสนอครั้งแรกในงานวิจัยชื่อ "Deep Residual Learning for Image Recognition" โดยมีจุดประสงค์เพื่อแก้ปัญหา Vanishing Gradient ซึ่งมักเกิดขึ้นในโครงข่ายที่มีความลึกมาก งานวิจัยนี้แนะนำการเพิ่ม "ทางลัด" หรือ shortcut connections ในโครงข่ายเพื่อให้สามารถข้ามผ่านเลเยอร์บางส่วนได้ ทำให้โครงข่ายสามารถเรียนรู้ได้ดีขึ้นแม้จะมีความลึกสูงมาก โดย ResNet ได้รับการอ้างอิงอย่างแพร่หลาย โดยมีการอ้างอิงในฐานข้อมูล Scopus ถึง 3,700 ครั้ง และใน Google Scholar ถึง 41,537 ครั้ง (ข้อมูลระหว่างเดือนมกราคม2016 ถึงมีนาคม2020) ถือเป็นหนึ่งในนวัตกรรมสำคัญที่ช่วยพัฒนาประสิทธิภาพของการจดจำภาพ

3.3.2 Vanishing Gradient

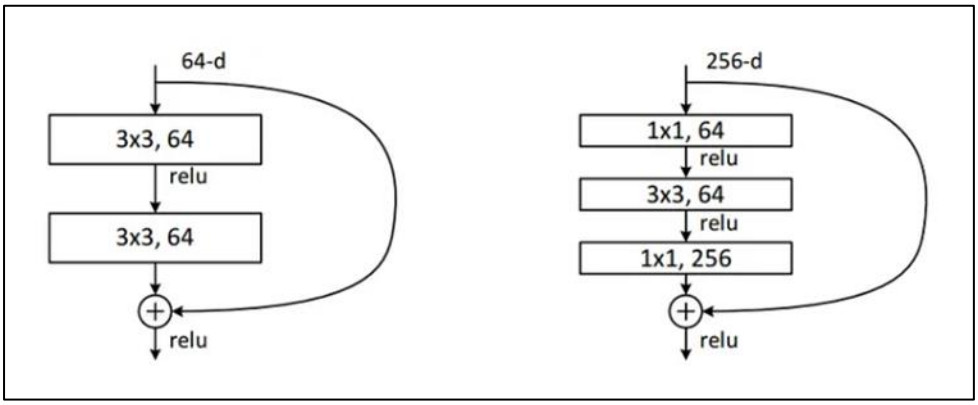
ปัญหาที่เกิดขึ้นจากการใช้เลเยอร์แบบคอนโวลูชันหลายชั้นคือ ผลลัพธ์จากเลเยอร์แรก (เช่น ค่าในอาเรย์ x) จะค่อย ๆ เลือนหายไปและยากที่จะติดตามว่า การเปลี่ยนแปลงค่าพารามิเตอร์ในเลเยอร์แรกส่งผลอย่างไร เนื่องจากผลลัพธ์จะถูกกลืนไปกับการประมวลผลในเลเยอร์ถัดมา

เราสามารถแก้ปัญหานี้ได้โดยการเพิ่มทางลัด (shortcut connection) ให้ค่า x ส่งผ่านข้ามชั้นไปยังเลเยอร์ที่สามโดยตรง และนำผลจากเลเยอร์ที่สามมาบวกกับ x วิธีนี้จะช่วยให้ค่าจากเลเยอร์ก่อนหน้าไม่สูญหาย และสามารถเรียนรู้ได้ดีขึ้นในโครงข่ายที่มีหลายเลเยอร์ ดังรูปที่ 3.9



รูปที่ 3.9 shortcut connection

อย่างไรก็ตามแต่การนำเลเยอร์มาบวกกัน ขนาดต้องเท่ากัน

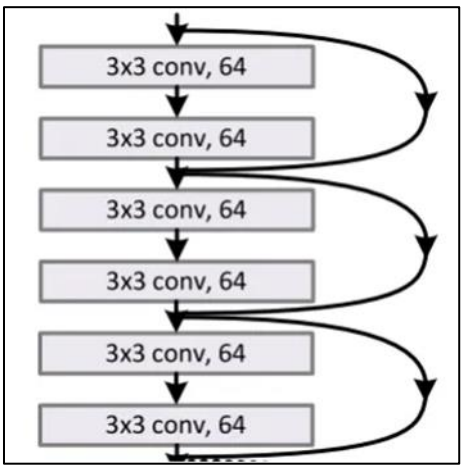


รูปที่ 3.10 Bottleneck Building Block

ในการพิจารณาโครงข่ายสองโครงสร้างในรูปที่ 3.10 ทางฝั่งซ้ายมีจำนวนฟีเจอร์แม็พเท่ากันที่ 64 ตลอดทั้งโครงข่าย ทำให้สามารถบวกกันได้ทันทีเมื่อมีการส่งผ่านค่าไปยังเลเยอร์ถัดไป ในขณะที่ทางฝั่งขวามีจำนวนฟีเจอร์แม็พเริ่มต้นที่ 256 ก่อนจะถูกลดลงเหลือ 64 ซึ่งต้องมีการปรับจำนวนฟีเจอร์แม็พเพื่อให้สอดคล้องกัน การปรับนี้สามารถทำได้โดยใช้ฟิลเตอร์ขนาด 1x1 เพื่อปรับจำนวนฟีเจอร์แม็พให้เหมาะสมก่อนบวกค่า

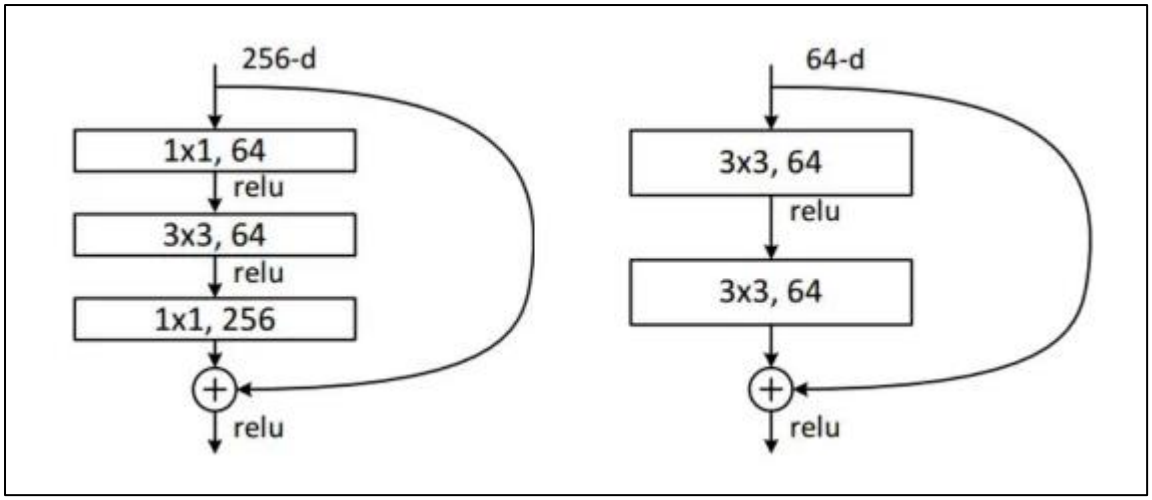
วิธีการปรับเปลี่ยนจำนวนฟีเจอร์แม็พแบบนี้เรียกว่า "Bottleneck Building Block" ซึ่งเป็นส่วนสำคัญในโครงสร้างของโครงข่าย ResNet

3.3.3 ResNet Architecture



รูปที่ 3.11 ResNet Block

โครงข่าย ResNet ประกอบด้วย 4 บล็อกใหญ่ (stages) ดังรูปที่ 3.11 โดยจำนวนเลเยอร์ที่มีพารามิเตอร์สำหรับการฝึกในโครงข่ายจะเป็นจำนวนชั้นที่ใช้เรียกชื่อ เช่น ResNet34 หมายถึงโครงข่ายที่มีจำนวนเลเยอร์ในบล็อกใหญ่รวมกัน 32 ชั้น บวกกับชั้นคอนโวลูชันที่ติดกับชั้นอินพุต และชั้น Dense ที่ติดกับชั้นเอาต์พุต เนื่องจาก ResNet มีโครงสร้างแบบคอนโวลูชันสองชั้นติดกันและมีเส้นทางลัด (shortcut connection) ข้ามชั้น เราจึงมักบอกจำนวนชั้นในแต่ละบล็อกใหญ่เป็นเลขหารสอง ตัวอย่างเช่น ใน ResNet34 บล็อกใหญ่แรกจะอธิบายขนาดเป็นค่า 3 ดังนั้น ResNet34 จะถูกอธิบายด้วยโครงสร้าง [3, 4, 6, 3] ซึ่งหมายถึงจำนวนคู่ในบล็อกทั้ง 4 โครงสร้างที่เป็นที่นิยมของ ResNet มักมีจำนวนชั้นเป็น 18, 34, 50, 101, และ 152



รูปที่ 3.12 Bottleneck block และ Normal block

แม้ ResNet จะมีโครงสร้างที่ประกอบด้วย 4 บล็อกใหญ่ และองค์ประกอบย่อยในแต่ละบล็อกมักจะมาเป็นคู่ ทำให้ดูเหมือนว่าโครงข่ายจะต้องมีจำนวนชั้นรวมเป็นเลขคู่ แต่เหตุผลที่ ResNet บางรุ่น เช่น ResNet101 มีจำนวนชั้นเป็นเลขคี่นั้นมาจากการใช้องค์ประกอบย่อยแบบ bottleneck ซึ่งประกอบด้วย 3 ชั้นแทนที่จะเป็นคู่ ดังรูปที่ 3.12 เมื่อจำนวนชั้นในโครงข่ายถึงระดับ 50 เราจะนิยมใช้องค์ประกอบแบบ bottleneck ทำให้ ResNet50 ถูกอธิบายขนาดเป็น [3, 4, 6, 3] ซึ่งคำนวณได้ (3+4+6+3)x3 = 48 ชั้น ส่วน ResNet101 มีโครงสร้าง [3, 4, 23, 3] ซึ่งคำนวณได้ (3+4+23+3)x3 = 99 ชั้น ชั้นภายนอกอีก 2 ชั้นจึงทำให้มีทั้งหมด 101 ชั้น ในทำนองเดียวกัน ResNet152 และ ResNet200 ก็ใช้โครงสร้างแบบ bottleneck ภายในเช่นกัน

3.4 Data Augmentation

Data Augmentation [10] เป็นเทคนิคที่ใช้ในการเพิ่มขนาดชุดข้อมูลการฝึกอบรม โดยการปรับแปลงข้อมูลที่มีอยู่แล้ว เทคนิคนี้มักถูกนำมาใช้ในงานด้านการเรียนรู้ของเครื่อง (Machine Learning) และการเรียนรู้เชิงลึก (Deep Learning) โดยเฉพาะในงานที่เกี่ยวข้องกับการประมวลผลภาพ (Computer Vision) เพื่อปรับปรุงความสามารถในการทั่วไป (Generalization) และความทนทาน (Robustness) ของโมเดลที่ฝึก

3.4.1 เหตุใดจึงควรใช้ Data Augmentation?

- เพิ่มขนาดชุดข้อมูล: ด้วยการสร้างตัวอย่างใหม่จากข้อมูลที่มีอยู่ การเพิ่มข้อมูลจะช่วยเพิ่มขนาดชุดข้อมูล ซึ่งสามารถนำไปสู่การปรับปรุงประสิทธิภาพของโมเดล
- การทำให้โมเดลมีความเป็นระเบียบ: การเพิ่มข้อมูลช่วยแนะนำความหลากหลายเพิ่มเติมให้กับข้อมูล ซึ่งสามารถช่วยป้องกันการเกิด Overfitting โดยทำให้โมเดลได้รับตัวอย่างที่หลากหลายมากขึ้น
- ปรับปรุงการทั่วไปของโมเดล: การทำให้โมเดลเผชิญกับความหลากหลายที่กว้างขึ้นในข้อมูล จะช่วยให้โมเดลสามารถทำงานได้ดีกับตัวอย่างที่ไม่เคยเห็นมาก่อน

3.4.2 เทคนิคที่ใช้ในการ Augmentation

- **Rotation:** หมุนภาพตามมุมต่าง ๆ เช่น 90 องศา หรือ 180 องศา

```
import Augmentor

# สร้าง pipeline สำหรับไฟล์ในโฟลเดอร์ที่มีภาพอยู่
pipeline = Augmentor.Pipeline("path_to_your_image_folder")

# Rotation: หมุนภาพตามมุมที่กำหนด (เช่น 90 องศา หรือ 180 องศา)
pipeline.rotate(probability=1.0, max_left_rotation=90, max_right_rotation=90)
```

รูปที่ 3.13 ตัวอย่างโค้ดการ Rotation

- **Translation:** เลื่อนภาพในแนวราบหรือแนวดิ่งเป็นระยะหนึ่ง

```
import Augmentor

# สร้าง pipeline สำหรับไฟล์ในโฟลเดอร์ที่มีภาพอยู่
pipeline = Augmentor.Pipeline("path_to_your_image_folder")

# Translation: เลื่อนภาพในแนวราบหรือแนวดิ่ง
pipeline.random_distortion(probability=1.0, grid_width=4, grid_height=4, magnitude=8)
```

รูปที่ 3.14 ตัวอย่างโค้ดการ Translation

- **Scaling:** ขยายหรือย่อภาพตามปัจจัยที่กำหนด

```
import Augmentor

# สร้าง pipeline สำหรับไฟล์ในโฟลเดอร์ที่มีภาพอยู่
pipeline = Augmentor.Pipeline("path_to_your_image_folder")

# Scaling: ขยายหรือย่อภาพตามปัจจัยที่กำหนด
pipeline.zoom_random(probability=1.0, percentage_area=0.8)  # ซูมเข้า 80% ของพื้นที่เดิม
```

รูปที่ 3.15 ตัวอย่างโค้ดการ Scailing

- **Flipping:** พลิกภาพในแนวนอนหรือแนวตั้ง

```
import Augmentor

# สร้าง pipeline สำหรับไฟล์ในโฟลเดอร์ที่มีภาพอยู่
pipeline = Augmentor.Pipeline("path_to_your_image_folder")

# Flipping: พลิกภาพในแนวนอนหรือแนวตั้ง
pipeline.flip_left_right(probability=0.5)  # พลิกภาพในแนวนอน
pipeline.flip_top_bottom(probability=0.5)  # พลิกภาพในแนวตั้ง
```

รูปที่ 3.16 ตัวอย่างโค้ดการ Flipping

- **Shearing:** ทำให้ภาพเอียงตามแกน x หรือ y

```
import Augmentor

# สร้าง pipeline สำหรับไฟล์ในโฟลเดอร์ที่มีภาพอยู่
pipeline = Augmentor.Pipeline("path_to_your_image_folder")

# Shearing: ทำให้ภาพเอียงตามแกน x หรือ y                # เอียง 10 องศาในแนว x หรือ y
pipeline.shear(probability=1.0, max_shear_left=10, max_shear_right=10)
```

รูปที่ 3.17 ตัวอย่างโค้ดการ Shearing

- **Zooming:** ซูมเข้า หรือซูมออกจากภาพ

```
import Augmentor

# สร้าง pipeline สำหรับไฟล์ในโฟลเดอร์ที่มีภาพอยู่
pipeline = Augmentor.Pipeline("path_to_your_image_folder")

# Zooming: ซูมเข้าในภาพ
pipeline.zoom(probability=1.0, min_factor=1.1, max_factor=1.5)  # ซูมเข้าได้ตั้งแต่ 1.1 ถึง 1.5 เท่า
```

รูปที่ 3.18 ตัวอย่างโค้ดการ Zooming

- **Brightness Adjustment:** เพิ่มหรือลดความสว่างของภาพ

```
import Augmentor

# สร้าง pipeline สำหรับไฟล์ในโฟลเดอร์ที่มีภาพอยู่
pipeline = Augmentor.Pipeline("path_to_your_image_folder")

# Brightness Adjustment: เพิ่มหรือลดความสว่างของภาพ          # สว่าง 0.5 ถึง 1.5 เท่าของภาพเดิม
pipeline.random_brightness(probability=1.0, min_factor=0.5, max_factor=1.5)
```

รูปที่ 3.19 ตัวอย่างโค้ดการ Brightness Adjustment

- **Contrast Adjustment:** เพิ่มหรือลดความคมชัดของภาพ

```
import Augmentor

# สร้าง pipeline สำหรับไฟล์ในโฟลเดอร์ที่มีภาพอยู่
pipeline = Augmentor.Pipeline("path_to_your_image_folder")

# Contrast Adjustment: เพิ่มหรือลดความคมชัดของภาพ          # ความคมชัด 0.5 ถึง 1.5 เท่า
pipeline.random_contrast(probability=1.0, min_factor=0.5, max_factor=1.5)
```

รูปที่ 3.20 ตัวอย่างโค้ดการ Contrast Adjustment

– **Noise Addition:** เพิ่มสัญญาณรบกวนแบบสุ่มลงในภาพ

```
import Augmentor

# สร้าง pipeline สำหรับไฟล์ในโฟลเดอร์ที่มีภาพอยู่
pipeline = Augmentor.Pipeline("path_to_your_image_folder")

# Noise Addition: เพิ่มสัญญาณรบกวนแบบสุ่มลงในภาพ
pipeline.random_color(probability=1.0, min_factor=0.5, max_factor=1.5)  # เพิ่มสีสันแบบสุ่ม
```

รูปที่ 3.21 ตัวอย่างโค้ดการทำ Noise Addition

3.5 Digital Image Processing

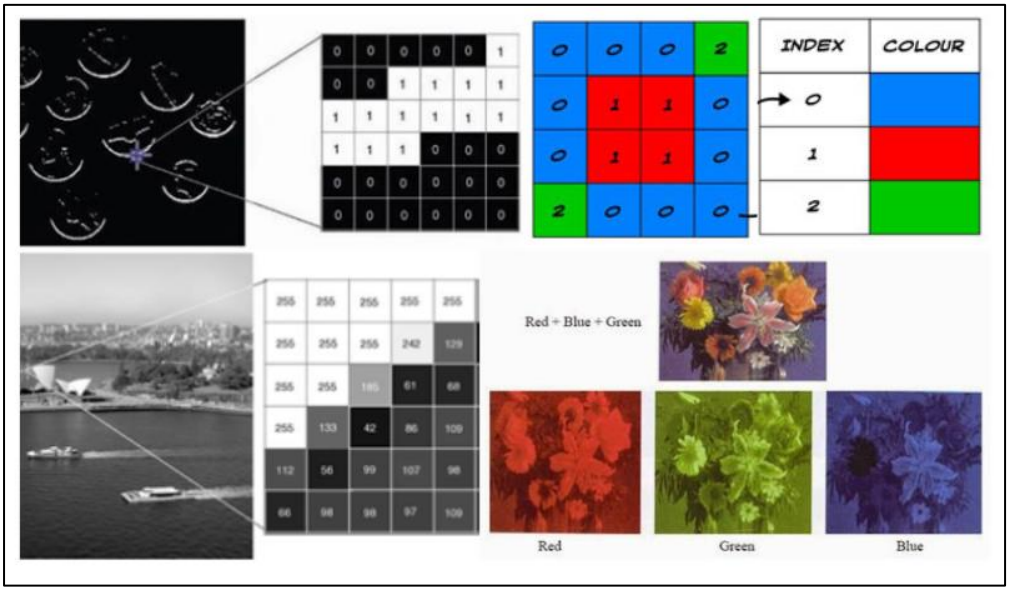
3.5.1 ภาพดิจิทัลคืออะไร

ภาพดิจิทัล[11]คือการแทนค่าของภาพในรูปแบบของพิกเซล ที่ประกอบกันเป็นภาพทั้งหมด พิกเซลเหล่านี้มีค่าความสว่างและสีที่แตกต่างกัน ทำให้เกิดเป็นภาพดิจิทัลที่เรามองเห็น

3.5.2 การแบ่งประเภทของภาพดิจิทัล

ภาพดิจิทัลสามารถแบ่งออกเป็น 4 ประเภทดังนี้ [11]

- **Binary Image** ภาพขาว-ดำ 1 บิตต่อพิกเซล ค่าสีมี 2 ค่า คือ 0 สีดำ 1 สีขาว เหมาะกับภาพที่เกี่ยวกับตัวอักษร ภาพลายนิ้วมือ
- **Grayscale Image** อาร์เรย์ 2 มิติ โดยค่าที่เก็บจะแตกต่างกันไป ตั้งแต่สีขาวไปยังสีดำ ภาพระดับสีเทา มีความละเอียด 8 บิต ค่าความเข้มของแสงสีดำ 0 ค่าความเข้มของแสงสีขาว 255
- **RGB Image หรือ Truecolor Image** อาร์เรย์ 3 มิติ ขนาด m x n x 3 โดยที่ m คือความยาว และ n คือความกว้างของภาพในหน่วยพิกเซล ส่วนมิติสุดท้ายนั้น ในแต่ละมิติจะเก็บค่าสีแยกกัน คือสีแดง (Red) สีเขียว (Green) และสีน้ำเงิน (Blue)
- **Indexed Image** มีรูปแบบการเก็บแบบ indexed คือ ภาพประเภทนี้จะเก็บ ค่าสีเป็น indexed และในแต่ละช่องอาร์เรย์ จะเก็บตำแหน่งของสีใน indexed นั้น ๆ ไว้



รูปที่ 3.22 Binary Image, Grayscale Image, RGB Image and Indexed Image

3.5.3 ขั้นตอนในการประมวลผลภาพ

การประมวลผลภาพดิจิทัล [12] ประกอบด้วยหลายขั้นตอน ซึ่งแต่ละขั้นตอนมีบทบาทในการปรับปรุงและวิเคราะห์ภาพดิจิทัลให้มีประสิทธิภาพและเหมาะสมกับการนำไปใช้ในงานต่าง ๆ ดังนี้:

- **Image Acquisition (การนำภาพเข้าระบบ):** เป็นขั้นตอนแรกที่นำภาพเข้าสู่ระบบ โดยภาพจะผ่านกระบวนการ preprocessing เพื่อทำความสะอาดหรือปรับแก้ก่อนเข้าสู่ขั้นตอนการประมวลผลต่อไป

```
import cv2
image = cv2.imread('image.jpg')
# โหลดภาพจากไฟล์ 'image.jpg' และเก็บในตัวแปร image
```

รูปที่ 3.23 ตัวอย่างโค้ดการนำภาพเข้าระบบ

- **Image Enhancement (การปรับปรุงภาพ):** ปรับปรุงภาพให้ชัดเจนขึ้น โดยเฉพาะกับภาพที่มีรายละเอียดไม่ชัดเจน เช่น การเพิ่มความคมชัดและการเพิ่มคอนทราสต์ เพื่อให้รายละเอียดในภาพเด่นชัดขึ้น

```
import cv2
image = cv2.imread('image.jpg')
# โหลดภาพ
enhanced_image = cv2.equalizeHist(image)
# ปรับคอนทราสต์ของภาพให้สมดุลเพื่อให้รายละเอียดชัดเจนขึ้น
```

รูปที่ 3.24 ตัวอย่างโค้ดการปรับคอนทราสต์

- **Image Restoration (การคืนภาพสู่สภาพเดิม):** ทำให้ภาพกลับคืนสู่สภาพที่ดีขึ้นหรือปรับภาพให้เหมาะสมกับการมองเห็นมากขึ้น โดยใช้วิธีลดสัญญาณรบกวน (Noise) และแก้ไขความบิดเบี้ยวในภาพ

```
import cv2
image = cv2.imread('noisy_image.jpg')
# โหลดภาพ
restored_image = cv2.GaussianBlur(image, (5, 5), 0)
# ใช้ Gaussian Blur เพื่อลด noise และทำให้ภาพเรียบเนียนขึ้น
```

รูปที่ 3.25 ตัวอย่างโค้ดลดสัญญาณรบกวน

- **Color Image Processing (การประมวลผลภาพสี):** ใช้รูปแบบสี เช่น RGB เพื่อแยกแยะคุณลักษณะของภาพและแยกภาพที่ต้องการ การประมวลผลภาพสีช่วยให้เข้าใจคุณสมบัติของสีและการแยกวัตถุที่สนใจในภาพ

```
import cv2
image = cv2.imread('color_image.jpg')
# โหลดภาพ
b, g, r = cv2.split(image)
# แยกสีออกเป็นสามแชนเนล (แดง เขียว น้ำเงิน)
```

รูปที่ 3.26 ตัวอย่างโค้ดการแยกแชแนลสี R G B

- **Wavelets and Multi-Resolution Processing:** เป็นการใช้เทคนิค Wavelet ในการประมวลผลภาพแบบหลายระดับ ซึ่งช่วยให้สามารถตรวจจับรายละเอียดต่าง ๆ ในภาพได้ในหลายระดับความละเอียด

```
import cv2
import pywt

image = cv2.imread('image.jpg')
# โหลดภาพ
coeffs = pywt.wavedec2(image, 'haar', level=2)
# ใช้ Wavelet decomposition แยกภาพออกเป็นหลายระดับความละเอียด
cA, (cH, cV, cD) = coeffs
# เก็บผลการแยกในตัวแปร cA (approximation), cH (horizontal), cV (vertical), cD (diagonal)
```

รูปที่ 3.27 ตัวอย่างโค้ดการใช้เทคนิค Wavelet

- **Compression (การบีบอัดภาพ):** ลดขนาดข้อมูลภาพลงโดยไม่ลดทอนคุณภาพมากเกินไป ทำให้สามารถเก็บข้อมูลภาพได้มากขึ้นในพื้นที่ที่จำกัด เช่น การบีบอัด JPEG

```
import cv2
image = cv2.imread('image.jpg')
# โหลดภาพต้นฉบับ
cv2.imwrite('compressed_image.jpg', image, [int(cv2.IMWRITE_JPEG_QUALITY), 50])
# บีบอัดภาพเป็น JPEG คุณภาพ 50%
```

รูปที่ 3.28 ตัวอย่างโค้ดการบีบอัด JPEG

- **Morphological Processing (การประมวลผลเชิงรูปร่าง):** ใช้ในการแยกส่วนประกอบของภาพเพื่อแสดงรูปร่างของวัตถุ เช่น การตรวจจับขอบและการวิเคราะห์รูปร่างต่าง ๆ ในภาพ การลบ noise ขนาดเล็กออกจากภาพ เช่น จุดสีขาวในภาพไบนารี

```
import cv2
import numpy as np

image = cv2.imread('binary_image.jpg', 0)
# โหลดภาพขาวดำแบบไบนารี
kernel = np.ones((5, 5), np.uint8)
# กำหนด kernel ขนาด 5x5
morphed_image = cv2.morphologyEx(image, cv2.MORPH_OPEN, kernel)
# ใช้การเปิดเพื่อลด noise
```

รูปที่ 3.29 ตัวอย่างโค้ดการลบจุดสีขาวในภาพไบนารี

- **Segmentation (การแบ่งส่วนของภาพ):** แยกส่วนที่สนใจออกมาจากภาพทั้งหมด เพื่อให้สามารถวิเคราะห์เฉพาะส่วนที่ต้องการ เช่น การแยกวัตถุหรือคนออกจากพื้นหลัง

```
import cv2

image = cv2.imread('image.jpg', 0)
# โหลดภาพขาวดำ
_, segmented_image = cv2.threshold(image, 128, 255, cv2.THRESH_BINARY)
# แปลงภาพเป็นไบนารีโดยใช้ threshold ที่ค่า 128
```

รูปที่ 3.30 ตัวอย่างโค้ดแบ่งส่วนของภาพด้วย thresholding เพื่อตัดภาพเป็นขาว-ดำ

- **Representation and Description (การแทนค่าภาพและการบรรยาย):** ใช้เพื่อแทนค่าลักษณะเฉพาะของวัตถุในภาพและบรรยายคุณสมบัติของวัตถุ เช่น รูปร่าง ขนาด และตำแหน่ง

```
import cv2

image = cv2.imread('image.jpg', 0)
# โหลดภาพขาวดำ
_, thresh = cv2.threshold(image, 128, 255, cv2.THRESH_BINARY)
# ทำ threshold เพื่อสร้างภาพไบนารี
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
# หา contours ในภาพ
cv2.drawContours(image, contours, -1, (0, 255, 0), 2)
# วาด contours บนภาพเดิม
```

รูปที่ 3.31 ตัวอย่างโค้ดการค้นหาและวาดขอบเขตของวัตถุที่อยู่ในภาพ

- **Object Recognition (การรู้จำวัตถุ):** เป็นขั้นตอนสุดท้ายที่ใช้ในการจำแนกหรือรู้จำวัตถุในภาพ เช่น การจดจำใบหน้า การรู้จำตัวอักษร หรือการตรวจจับสิ่งของ

```
import cv2

face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
# โหลดข้อมูลการตรวจจับใบหน้า
image = cv2.imread('people.jpg')
# โหลดภาพที่มีใบหน้า
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# แปลงภาพเป็นขาวดำเพื่อการตรวจจับง่ายขึ้น

faces = face_cascade.detectMultiScale(gray, 1.1, 4)
# ตรวจจับใบหน้าในภาพโดยกำหนดขนาดการสแกน
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)
# วาดกรอบรอบใบหน้าที่ตรวจจับได้
```

รูปที่ 3.32 ตรวจจับใบหน้าในภาพโดยใช้ Haar Cascade

บทที่ 4 ขั้นตอนและแผนการดำเนินงาน

ในบทนี้จะกล่าวถึงขั้นตอนและการวางแผนการดำเนินงานและระยะเวลาที่ใช้ในการทำโครงงานวิจัย

4.1 ขั้นตอนการดำเนินงาน

4.1.1 กำหนดหัวข้อโครงงาน

- เลือกหัวข้อโครงงาน
- รวบรวมข้อมูลที่เกี่ยวข้อง

4.1.2 กำหนดขอบเขตและวางแผนกำเนินงาน

- กำหนดขอบเขตโครงงาน
- วางแผนการดำเนินงาน

4.1.3 ศึกษาความรู้และทฤษฎีที่เกี่ยวข้อง

- ศึกษาเกี่ยวกับประติมากรรมลวดลายปูนปั้นและความรู้ที่เกี่ยวข้อง
- Convolution Neural Network (CNN)
- HTML, CSS, JavaScript
- Flask

4.1.4 ออกแบบระบบ

- ออกแบบหน้าต่างส่วนผู้ใช้งาน ให้สามารถติดต่อกับผู้ใช้งานได้

4.1.5 พัฒนาระบบ

- รวบรวมภาพในการพัฒนาโมเดล
- สร้างตัวโมเดลในการจำแนกภาพ
- สร้างหน้าเว็บในการใช้งานแสดงผลของโมเดล
- เชื่อมต่อระหว่างหน้าเว็บกับส่วนของโมเดล

4.1.6 ทดสอบและปรับปรุง

- ทดสอบความสามารถในการใชงานโปรแกรม
- ตรวจสอบความผิดพลาดของโปรแกรม และแก้ไขข้อผิดพลาด

4.1.7 จัดทำเอกสารประกอบโครงงาน

- จัดทำรายละเอียดของโครงงาน ตั้งแต่ขั้นตอนเริ่มต้น จนถึงขั้นตอนสุดท้ายของโครงงาน

4.2 แผนการดำเนินงาน

ตารางระบุระยะเวลาในการดำเนินงานของแต่ละขั้นตอนการดำเนินงานมีดังนี้

ตารางที่ 4.1 แผนและขั้นตอนการดำเนินงาน

ขั้นตอนการดำเนินงาน	พ.ย. 2566	ธ.ค. 2566	ม.ค. 2567	ก.พ. 2567	มี.ค. 2567	ม.ย. 2567	พ.ค. 2567	มิ.ย. 2567	ก.ค. 2567	ส.ค. 2567	ก.ย. 2567	ต.ค. 2567	พ.ย. 2567
1.คิดหัวข้อและเสนอโครงงานวิจัย	↔												
2.วางแผนการดำเนินการ		↔	↔	↔									
3.ศึกษาทฤษฎีความรู้ที่เกี่ยวข้อง		↔	↔	↔	↔	↔	↔	↔	↔				
4.รวบรวมข้อมูล					↔	↔	↔	↔	↔	↔			
5.สร้างโมเดลสำหรับการทำนาย						↔	↔	↔	↔	↔	↔		
6.ออกแบบและทำการทดสอบ Web Application					↔	↔	↔	↔	↔	↔	↔		
7.จัดทำเอกสารประกอบโครงงานวิจัย		↔	↔	↔	↔	↔	↔	↔	↔	↔	↔	↔	↔

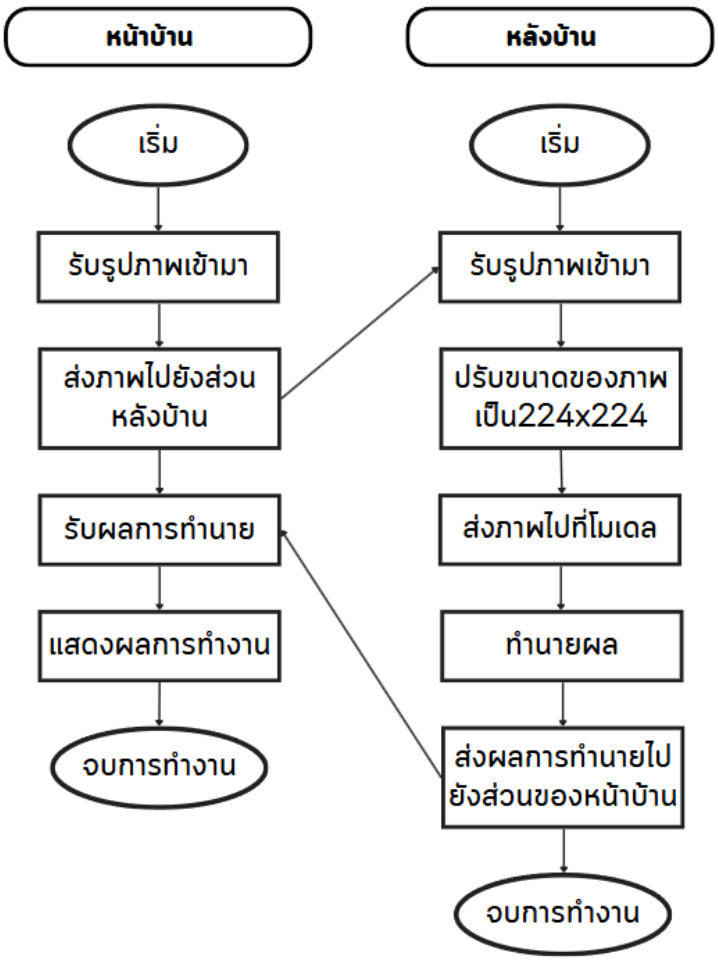
บทที่ 5 การวิเคราะห์และออกแบบระบบ

ในบทนี้จะกล่าวถึงโครงสร้าง และการทำงานของระบบ รวมถึงกระบวนการทำงานของระบบว่ามีลักษณะการทำงานอย่างไร ซึ่งมีลักษณะดังต่อไปนี้

5.1 โครงสร้างของระบบ

มี 2 ส่วนคือ ส่วนหน้าบ้าน และ ส่วนหลังบ้าน

5.1.1 ส่วนการทำงานของเว็บแอปพลิเคชัน



รูปที่ 5.1 โฟลวชาร์ตการทำงานของเว็บแอปพลิเคชัน

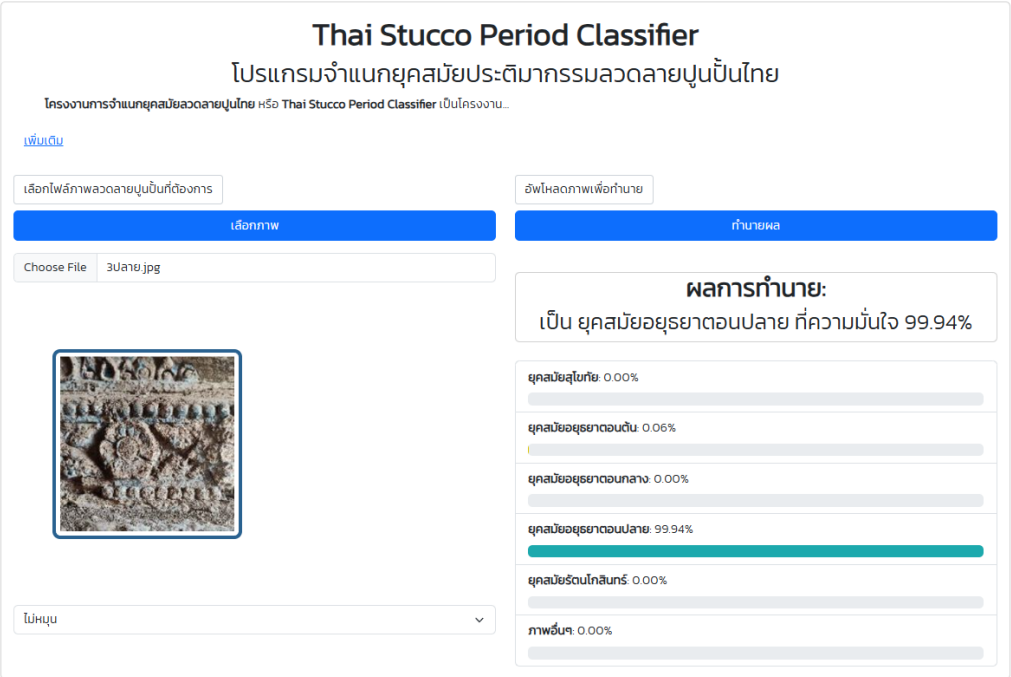
กระบวนการทำงานของระบบเริ่มต้นที่ส่วนหน้าบ้าน เมื่อระบบได้รับภาพเข้ามาจากผู้ใช้ ภาพนี้จะถูกส่งต่อไปยังส่วนหลังบ้านเพื่อทำการประมวลผล ในส่วนหลังบ้าน ภาพจะถูกปรับขนาดให้เป็น 224x224 พิกเซล ซึ่งเป็นขนาดที่โมเดลต้องการ หลังจากปรับขนาดภาพแล้ว ภาพจะถูกส่งต่อไปยังโมเดลเพื่อทำการทำนายผล เมื่อโมเดลทำนายเสร็จแล้ว ระบบจะได้รับผลการทำนายจากโมเดลนั้น และส่งผลนี้กลับไปยังส่วนหน้าบ้าน เพื่อแสดงผลให้ผู้ใช้เห็นผ่านหน้าเว็บแอปพลิเคชันที่จัดทำไว้

5.1.2 หน้าส่วนของผู้ใช้งาน

5.1.2.1 การออกแบบหน้าส่วนของผู้ใช้งาน

องค์ประกอบหลัก ๆ มีดังนี้:

1. **ชื่อโปรแกรม**: "Thai Stucco Period Classifier" แสดงอยู่ด้านบนสุดพร้อมคำอธิบายเป็นภาษาไทย เพื่อให้ผู้ใช้งานเข้าใจว่าโปรแกรมนี้มีจุดประสงค์ในการจำแนกยุคสมัยของประติมากรรมลวดลายปูนปั้นไทย
2. **ลิงก์ "เพิ่มเติม"**: ลิงก์นี้อยู่ใต้คำอธิบายโครงการ ใช้เพื่อแสดงรายละเอียดเพิ่มเติมเกี่ยวกับโครงงานและปูนปั้น
3. **ส่วนการอัปโหลดภาพ**: มีข้อความแนะนำ "เลือกไฟล์ภาพลวดลายปูนปั้นที่ต้องการ" และปุ่ม "เลือกภาพ" ที่ช่วยให้ผู้ใช้สามารถเลือกภาพจากอุปกรณ์ของตนเอง
4. **ส่วนการทำนายผล**: เมื่อภาพถูกอัปโหลดเรียบร้อยแล้ว ผู้ใช้สามารถกดปุ่ม "ทำนายผล" เพื่อสั่งให้ระบบเริ่มทำการทำนายยุคสมัยของลายปูนปั้นที่อัปโหลดมา
5. **แสดงภาพที่เลือก**: มีการแสดงภาพที่ผู้ใช้ได้เลือกและอัปโหลดเข้ามาไว้ในระบบ เพื่อให้ผู้ใช้สามารถตรวจสอบและยืนยันว่าภาพที่เลือกนั้นถูกต้องก่อนที่จะดำเนินการทำนายผล
6. **ดรอปดาวน์การหมุน**: ที่อยู่ใต้ไฟล์ภาพใน นี้ใช้สำหรับหมุนภาพที่อัปโหลด โดยมีตัวเลือกให้ผู้ใช้หมุนภาพเป็น 90, 180, และ 270 องศา
7. **กล่องแสดงผลลัพธ์ที่มีความมั่นใจสูงสุด**: ในกล่องนี้จะแสดงข้อมูล 2 อย่างคือ ยุคสมัยของลายปูนปั้น และค่าความมั่นใจ
8. **กล่องแสดงหลอดค่าความมั่นใจทั้งหมด:** เป็นส่วนที่ออกแบบมาเพื่อแสดงข้อมูล 3 อย่าง คือ ชื่อยุคสมัย, หลอดค่าความมั่นใจ และเปอร์เซ็นต์ความมั่นใจ



รูปที่ 5.2 หน้าส่วนของผู้ใช้งาน

5.1.3 ที่มาของ Dataset ภาพสี

ในกระบวนการสร้าง Dataset สำหรับการจำแนกภาพลวดลายปูนปั้นจากภาพอื่น ๆ เราได้ดำเนินการลงพื้นที่ถ่ายภาพในแหล่งประวัติศาสตร์สำคัญต่าง ๆ ของประเทศไทย และแบ่งกลุ่มภาพตามยุคสมัยดังนี้

- ยุคสุโขทัย ได้มา 176 ภาพ
- อยุธยาตอนต้น เก็บรวบรวมมา 444 ภาพ
- อยุธยาตอนกลาง มีจำนวน 203 ภาพ
- อยุธยาตอนปลาย ได้จำนวน 584 ภาพ
- รัตนโกสินทร์ ได้รวบรวม 350 ภาพ

นอกจากนี้ เพื่อให้โมเดลสามารถแยกแยะภาพลวดลายปูนปั้นออกจากภาพอื่น ๆ ได้อย่างแม่นยำ เราได้ใช้ข้อมูลภาพจาก Kaggle ซึ่งประกอบด้วยภาพประเภทอื่น ๆ ที่ไม่เกี่ยวข้องกับลวดลายปูนปั้น

- ภาพอื่น ๆ 4,254 ภาพ

5.1.4 การ Augmentation

การใช้ Image Augmentation มีความสำคัญมากในงานด้านคอมพิวเตอร์วิชั่น (Computer Vision) โดยเฉพาะในการสร้างแบบจำลองปัญญาประดิษฐ์ (AI) และ Machine Learning เนื่องจากช่วยเพิ่มคุณภาพและปริมาณข้อมูลสำหรับการฝึกโมเดล

เทคนิคการ Augmentation ที่ใช้ใน Dataset คือ

- **หมุนภาพ (Rotation)**
  - หมุนภาพไปทางซ้ายหรือขวาแบบสุ่มภายในช่วง -10 องศาถึง 10 องศา ด้วยความน่าจะเป็น 70%
- **พลิกภาพแนวนอน (Horizontal Flip)**
  - พลิกภาพทางซ้ายขวาแบบสุ่ม ด้วยความน่าจะเป็น 50%
- **พลิกภาพแนวตั้ง (Vertical Flip)**
  - พลิกภาพขึ้นลงแบบสุ่ม ด้วยความน่าจะเป็น 50%
- **ซูม (Zoom)**
  - ซูมแบบสุ่ม โดยให้แสดงเฉพาะ 80% ของพื้นที่ภาพ ด้วยความน่าจะเป็น 50%
- **ปรับความต่างของแสง (Contrast)**
  - ปรับความต่างของแสงแบบสุ่มในช่วง 0.7 ถึง 1.3 ของค่าเดิม ด้วยความน่าจะเป็น 50%
- **ปรับความสว่าง (Brightness)**
  - ปรับความสว่างแบบสุ่มในช่วง 0.7 ถึง 1.3 ของค่าเดิม ด้วยความน่าจะเป็น 50%

```
p = Augmentor.Pipeline(image_folder)

p.rotate(probability=0.7, max_left_rotation=10, max_right_rotation=10)
p.flip_left_right(probability=0.5)
p.flip_top_bottom(probability=0.5)
p.zoom_random(probability=0.5, percentage_area=0.8)
p.random_contrast(probability=0.5, min_factor=0.7, max_factor=1.3)
p.random_brightness(probability=0.5, min_factor=0.7, max_factor=1.3)
```

รูปที่ 5.3 โค้ดที่ใช้ในการทำ Augmentation

รูปที่ 5.4 ตัวอย่างก่อนและหลังทำการ Augmentation

5.1.5 ที่มาของ Dataset ขาวเทา

ในส่วนของภาพ Dataset ขาวเทา เกิดจากนำ Dataset ภาพสีทำการ ใช้ไลบรารี่ pillow ในการ convert ภาพทั้งหมดเป็นขาวเทา

```
for filename in os.listdir(folder_path):
    if filename.endswith(('.png', '.jpg', '.jpeg', '.bmp', '.tiff')):
        img_path = os.path.join(folder_path, filename)
        img = Image.open(img_path).convert('L')

        output_path = os.path.join(output_folder_path, filename)
        img.save(output_path)

        print(f"Converted {filename} to grayscale.")
```

รูปที่ 5.5 โค้ดที่ใช้ในการแปลงภาพสีเป็นขาวเทา

5.2 กระบวนการทำงานทดลองและออกแบบโมเดล

ในการศึกษาทดลองหาโมเดลเพื่อนำมาใช้จำแนกยุคสมัยของประติมากรรมลวดลายปูนปั้น ได้ศึกษาการทำงานการปรับแต่งโมเดลเพื่อให้ได้ผลลัพธ์ที่มีประสิทธิภาพในการใช้งาน โดยโมเดลจะมีการฝึกฝนโดยใช้รูปภาพที่เป็นภาพสี และ ภาพขาวเทา

ในการทดลองนี้ มีการใช้โมเดลสองแบบ ได้แก่ CNN และ ResNet50 สำหรับโมเดล CNN นั้นจะถูกนำมาทดลองในสองรูปแบบของข้อมูลภาพ คือ ภาพสี และ ภาพขาวเทา และจะทดลองในขนาด kernel ที่ต่างกัน 3 ขนาดคือ 5x5, 11x11 และ 15x15 โดยทำการ train โมเดล 25 รอบ ด้วยค่าอัตราการเรียนรู้ 0.0001

ส่วนของโมเดล ResNet50 นั้นจะนำมาทดลองกับ ภาพสี และ ภาพขาวเทา เช่นเดียวกับ CNN เพื่อให้สามารถเปรียบเทียบผลลัพธ์จากสองโมเดลได้โดยตรง

5.2.1 การออกแบบโมเดล

โมเดล CNN ที่ใช้ในการทดลองโมเดลนี้เป็น CNN สำหรับการจำแนกภาพ โดยเริ่มจากการปรับค่าสเกลพิกเซล โดยมีทั้งหมด 3 เลเยอร์ จำนวนฟิลเตอร์ในแต่ละเลเยอร์ คือ 16, 32 และ64 และ MaxPooling เพื่อลดขนาดภาพและสกัดคุณลักษณะ จากนั้นข้อมูลจะถูกทำให้เป็น 1 มิติด้วย Flatten ชั้น Dropout (50%) สุดท้ายมี Dense layers สองชั้น (32 หน่วยและจำนวนหมวดหมู่ len(data_cat)) ชั้นสุดท้ายจะให้ผลลัพธ์การจำแนก โดยใช้ optimizer เป็น Adam และ loss function เป็นSparseCategorical

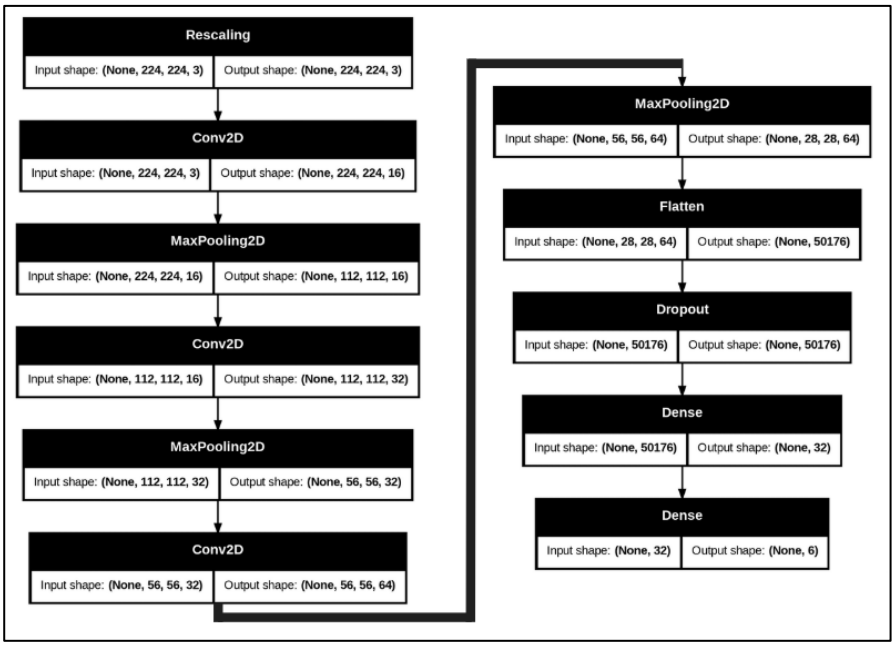
Crossentropy โดยปรับขนาด kernel ที่ ตัวแปร kernelSize ในโมเดล CNN ดังที่แสดงในรูปที่ 5.6 และการทดลองนี้ฝึกโมเดลด้วยข้อมูลฝึก (data_train) และข้อมูลประเมิน (data_eval) ทั้งหมด 25 epochs โดยบันทึกประวัติการฝึกใน history ดังรูปที่ 5.7 โดยรูปที่ 5.8 นั้นแสดงโครงสร้างของ โดยแสดงลำดับชั้นต่าง ๆ ของโมเดล ตั้งแต่ชั้น Rescaling เพื่อปรับสเกลพิกเซล, ชั้น Convolutional (Conv2D) และ MaxPooling เพื่อสกัดคุณลักษณะและลดขนาดภาพ, ชั้น Flatten เพื่อนำข้อมูลไปใช้ในชั้น Dense ซึ่งเป็นชั้น Fully Connected สุดท้ายในการจำแนกประเภท

```
input_shape = (224,224,3)
kernelSize = 5

model = Sequential()

model.add(Rescaling(1./255))
model.add(Conv2D(16, (kernelSize,kernelSize),padding='same',activation='relu',input_shape=input_shape))
model.add(MaxPooling2D())

model.add(Conv2D(32, (kernelSize,kernelSize),padding='same',activation='relu'))
model.add(MaxPooling2D())

model.add(Conv2D(64, (kernelSize,kernelSize),padding='same',activation='relu'))
model.add(MaxPooling2D())

model.add(Flatten())
model.add(Dropout(0.5))
model.add(Dense(32))#128
model.add(Dense(len(data_cat)))
opt = keras.optimizers.Adam(learning_rate=0.0001)
model.compile(optimizer=opt,loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),metrics=['accuracy'])
```

รูปที่ 5.6 โมเดล CNN ในการทดลอง

```
epochs_size = 25
history =  model.fit(data_train,validation_data = data_eval,epochs=epochs_size)
```

รูปที่ 5.7 จำนวน epoch และการ fit

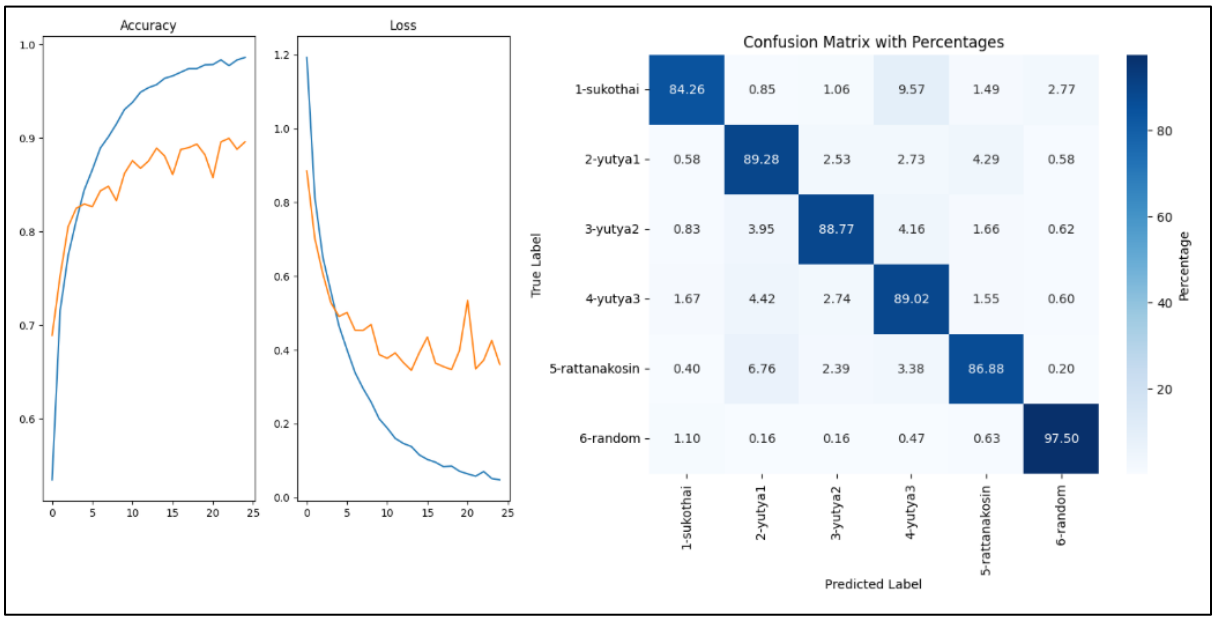


รูปที่ 5.8 โครงสร้างของโมเดล

5.2.2 ภาพสี

โมเดลภาพสีจะมีการทดลองทั้งหมด 4 โมเดล ได้แก่ CNN kernel 5x5, 11x11, 15x15 และ ResNet50 ในการทดลองจะกำหนดค่า inputshape=(224,224) และทำการฝึกฝนโมเดล 25 รอบ ค่าอัตราการเรียนรู้ 0.0001

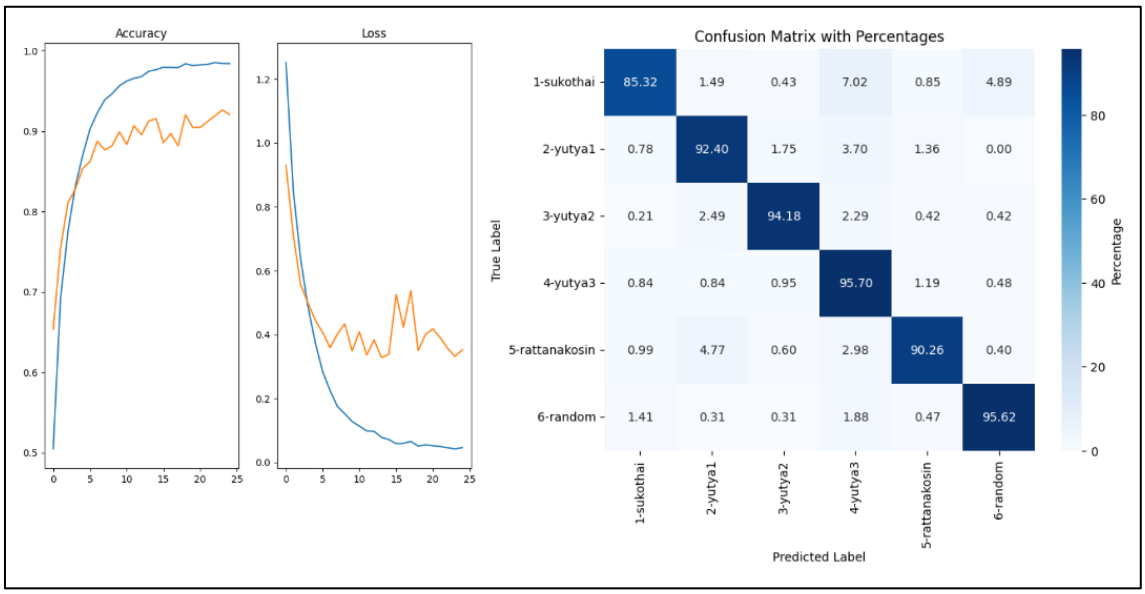
5.2.2.1 CNN kernel 5x5



รูปที่ 5.9 ความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล CNN kernel 5x5 ภาพสี

จากกราฟในรูปที่ 5.5 (โดยเส้นกราฟสีน้ำเงินคือการ Train และเส้นกราฟสีส้มคือการ Validation) การฝึกของโมเดล CNN kernel 5x5 ภาพสี แสดงให้เห็นว่าโมเดลสามารถเข้าใจข้อมูลฝึกได้ดี เนื่องจากค่าความแม่นยำในการฝึกเพิ่มขึ้นต่อเนื่อง และค่า Loss ลดลงอย่างมีนัยสำคัญ อย่างไรก็ตาม ในข้อมูลทดสอบพบว่าค่าความแม่นยำมีความผันผวนและค่า Loss ไม่ลดลงอย่างสม่ำเสมอ ซึ่งอาจชี้ให้เห็นถึงการเกิดปัญหา overfitting

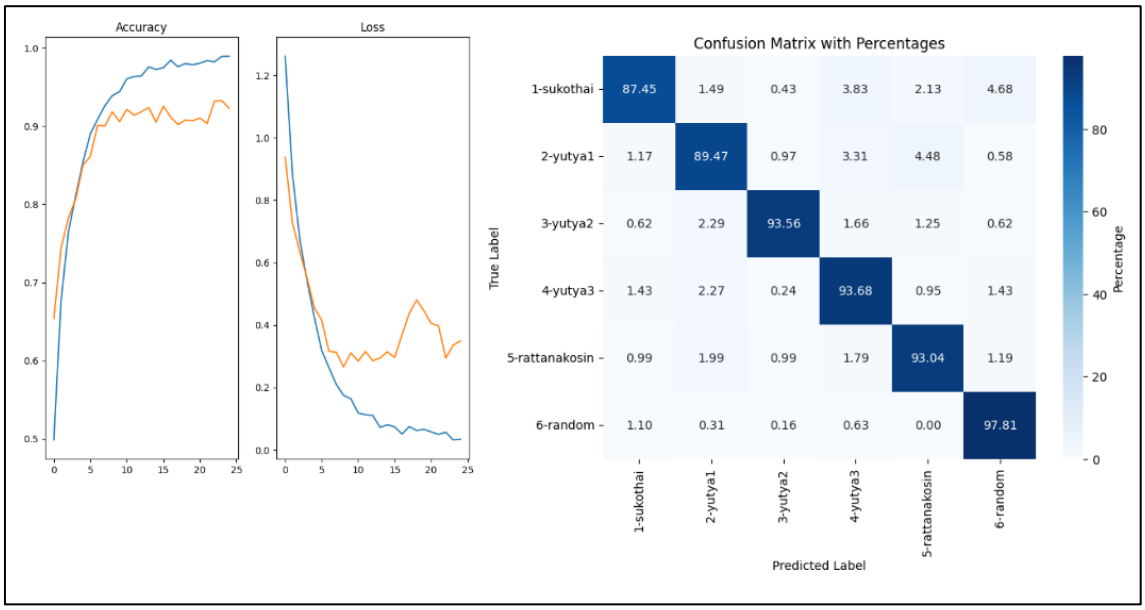
5.2.2.2 CNN kernel 11x11



รูปที่ 5.10 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล CNN kernel 11x11 ภาพสี

จากกราฟในรูปที่ 5.6 (โดยเส้นกราฟสีน้ำเงินคือการ Train และเส้นกราฟสีส้มคือการ Validation) การฝึกของโมเดล CNN kernel 11x11 ภาพสี แสดงให้เห็นว่าโมเดลยังสามารถเข้าใจข้อมูลฝึกได้ดี อย่างไรก็ตาม ในข้อมูลทดสอบพบว่าค่าความแม่นยำมีความผันผวนและค่า Loss ไม่ลดลงอย่างสม่ำเสมอ แต่ก็ดีขึ้นกว่าของโมเดล CNN kernel 5x5 ภาพสี ดังรูปที่ 5.5 ทั้งค่าความแม่นยำและค่า Loss แต่ลักษณะของกราฟ ก็ยังแสดงถึงการ overfitting

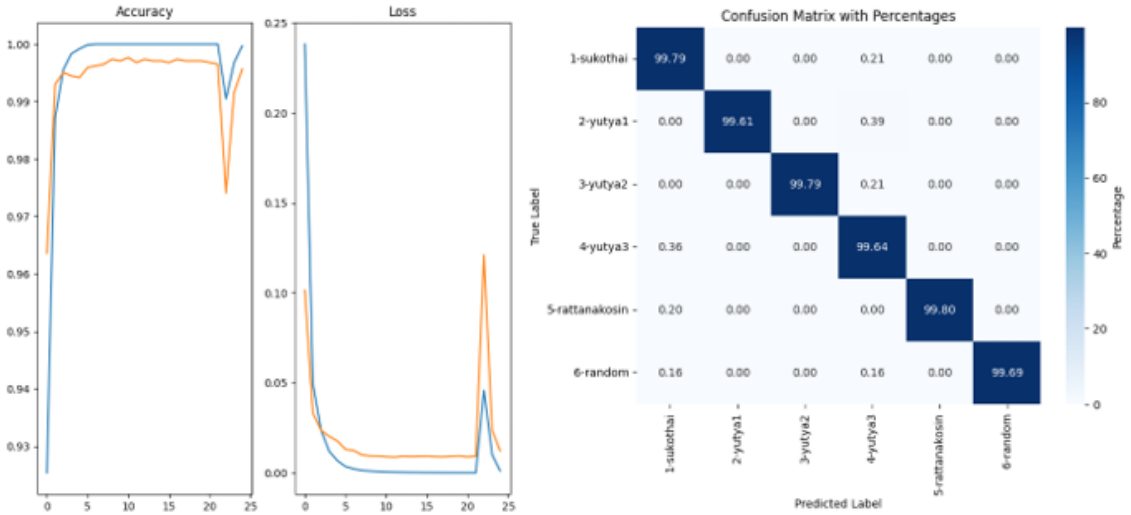
5.2.2.3 CNN kernel 15x15



รูปที่ 5.11 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล CNN kernel 15x15 ภาพสี

จากกราฟในรูปที่ 5.7 (โดยเส้นกราฟสีน้ำเงินคือการ Train และเส้นกราฟสีส้มคือการ Validation) การฝึกของโมเดล CNN kernel 15x15 ภาพสี แสดงให้เห็นว่าโมเดลยังสามารถเข้าใจข้อมูลฝึกได้ดี อย่างไรก็ตาม ในข้อมูลทดสอบพบว่าค่าความแม่นยำมีความผันผวนและค่า Loss ไม่ลดลงอย่างสม่ำเสมอ แต่ก็ดีขึ้นกว่าทั้งของโมเดล CNN kernel 5x5 ภาพสี ดังรูปที่ 5.5 และ CNN kernel 11x11 ภาพสี ดังรูปที่ 5.7 ทั้งค่าความแม่นยำและค่า Loss แต่ลักษณะของกราฟ ก็ยังแสดงถึงการ overfitting

5.2.2.4 ResNet50



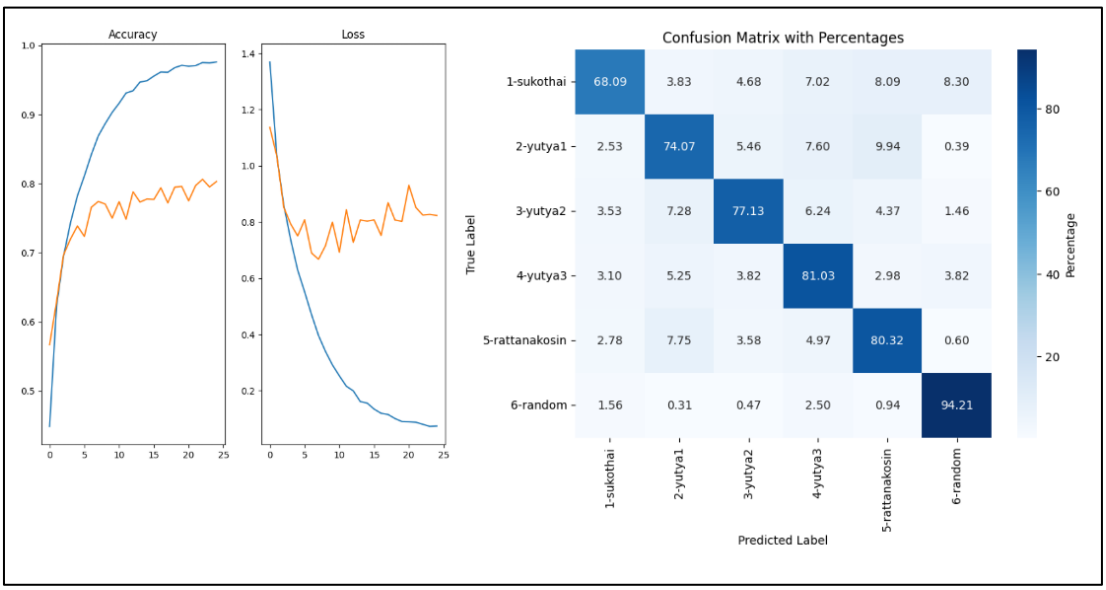
รูปที่ 5.12 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล ResNet50 ภาพสี

จากกราฟในรูปที่ 5.8 (โดยเส้นกราฟสีน้ำเงินคือการ Train และเส้นกราฟสีส้มคือการ Validation) การฝึกของโมเดล ResNet50 ภาพสีแสดงให้เห็นว่าโมเดลสามารถเข้าใจข้อมูลฝึกได้ดี และข้อมูลทดสอบเองก็ทำได้ดีมากเช่นเดียวกัน

5.2.1 ภาพขาวเทา

โมเดลภาพขาวเทาจะมีการทดลองทั้งหมด 4 โมเดล ได้แก่ CNN kernel 5x5, 11x11, 15x15 และ ResNet50 ในการทดลองจะทำการ แปลงภาพทั้งหมดในดาต้าเซ็ทภาพสี ให้เป็น ภาพ grayscale กำหนดค่า inputshape=(224,224) และทำการฝึกฝนโมเดล 25 รอบ ค่าอัตราการเรียนรู้ 0.0001

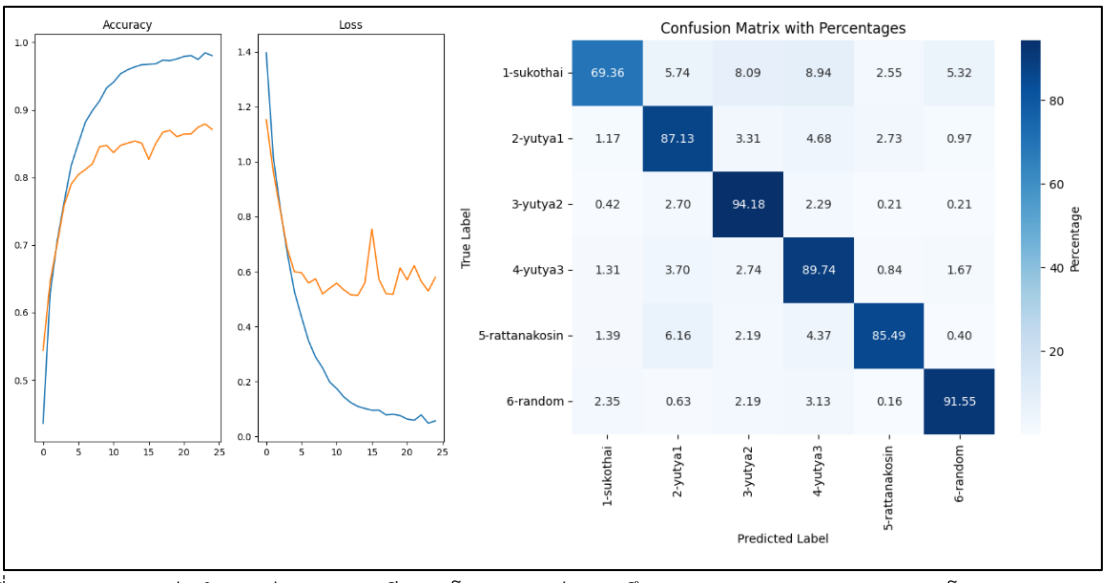
5.2.1.1 CNN kernel 5x5



รูปที่ 5.13 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล CNN kernel 5x5 ภาพขาวเทา

จากกราฟในรูปที่ 5.9 (โดยเส้นกราฟสีน้ำเงินคือการ Train และเส้นกราฟสีส้มคือการ Validation) การฝึกของโมเดล CNN kernel 5x5 ภาพขาวเทา แสดงให้เห็นว่าโมเดลสามารถเข้าใจข้อมูลฝึกได้ดี เนื่องจากค่าความแม่นยำในการฝึกเพิ่มขึ้นต่อเนื่อง และค่า Loss ลดลงอย่างมีนัยสำคัญ อย่างไรก็ตาม ในข้อมูลทดสอบพบว่าค่าความแม่นยำมีความผันผวนและค่า Loss ไม่ลดลงอย่างสม่ำเสมอ ซึ่งอาจชี้ให้เห็นถึงการเกิดปัญหา overfitting หากเทียบกับกราฟของ CNN kernel 5x5 ภาพสี ในรูปที่ 5.5 แล้ว จะเห็นได้ชัดว่าค่าความแม่นยำของโมเดล CNN kernel 5x5 ภาพขาวเทา น้อยกว่าอย่างชัดเจน และค่า Loss มากกว่าอย่างชัดเจน

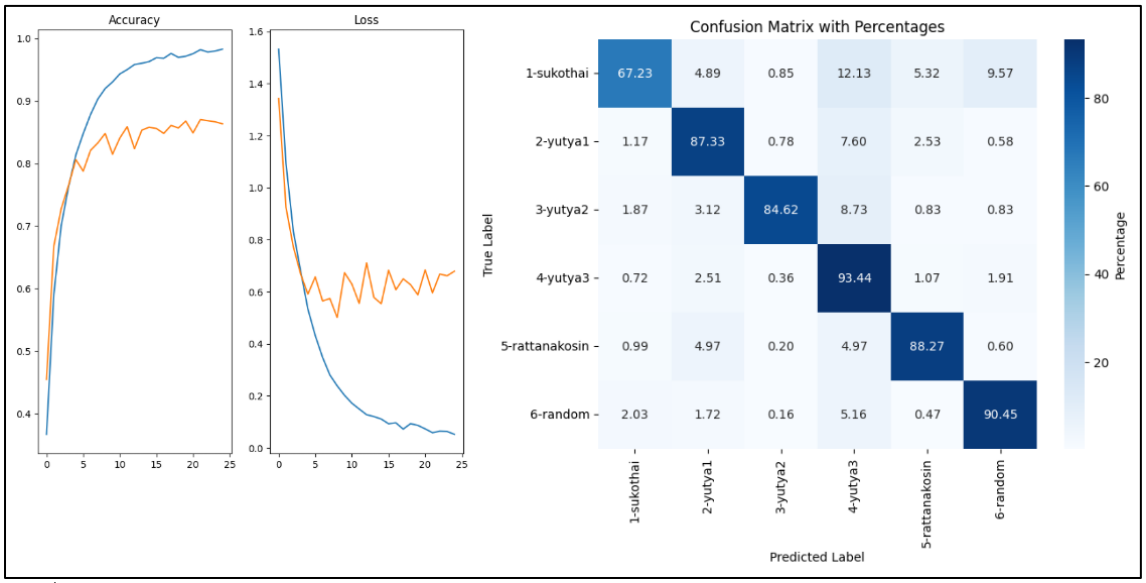
5.2.1.2 CNN kernel 11x11



รูปที่ 5.14 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล CNN kernel 11x11 ภาพขาวเทา

จากกราฟในรูปที่ 5.10 (โดยเส้นกราฟสีน้ำเงินคือการ Train และเส้นกราฟสีส้มคือการ Validation) การฝึกของโมเดล CNN kernel 11x11 ภาพขาวเทา แสดงให้เห็นว่าโมเดลยังสามารถเข้าใจข้อมูลฝึกได้ดี อย่างไรก็ตาม ในข้อมูลทดสอบพบว่าค่าความแม่นยำมีความผันผวนและค่า Loss ไม่ลดลงอย่างสม่ำเสมอ แต่ก็ดีขึ้นกว่าของโมเดล CNN kernel 5x5 ภาพขาวเทาจากรูปที่ 5.9 ทั้งค่าความแม่นยำและค่า Loss แต่ก็ยังด้อยกว่าอย่างชัดเจนเมื่อเทียบกับโมเดล CNN kernel 5x5 ภาพสี ดังรูปที่ 5.5 และลักษณะของกราฟกราฟในรูปที่ 5.10 ก็ยังแสดงถึงการ overfitting

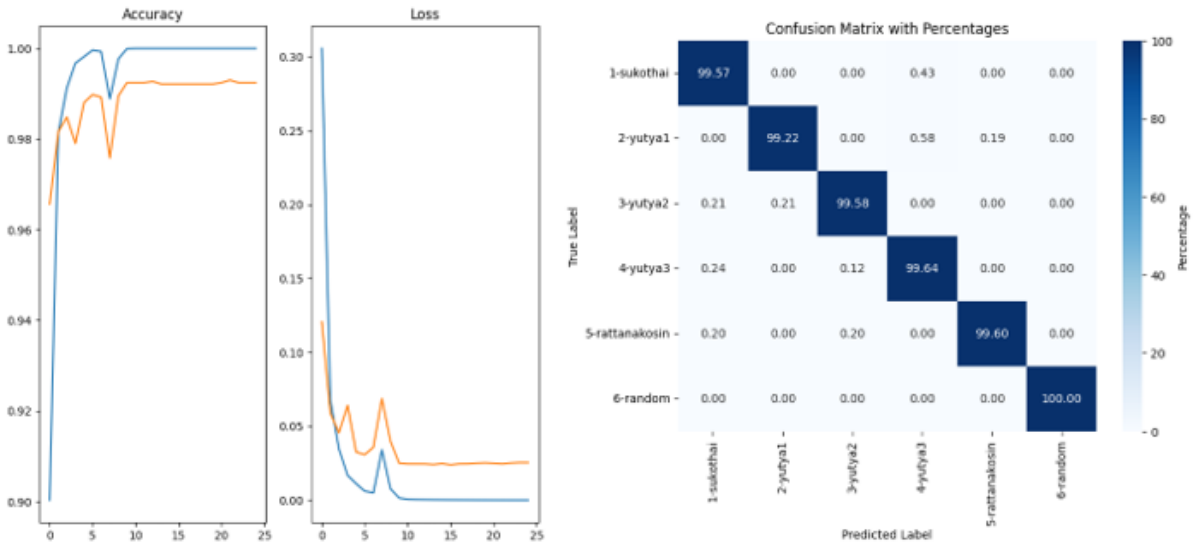
5.2.1.3 CNN kernel 15x15



รูปที่ 5.15 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล CNN kernel 15x15 ภาพขาวเทา

จากกราฟในรูปที่ 5.11 (โดยเส้นกราฟสีน้ำเงินคือการ Train และเส้นกราฟสีส้มคือการ Validation) การฝึกของโมเดล CNN kernel 15x15 ภาพขาวเทา แสดงให้เห็นว่าโมเดลยังสามารถเข้าใจข้อมูลฝึกได้ดี อย่างไรก็ตาม ในข้อมูลทดสอบพบว่าค่าความแม่นยำมีความผันผวนและค่า Loss ไม่ลดลงอย่างสม่ำเสมอ และอีกทั้งยังแย่ลงกว่าของโมเดล CNN kernel 11x11 ภาพขาวเทาดังรูปที่ 5.10 ทั้งค่าความแม่นยำและค่า Loss และ ก็ยังแสดงถึงการ overfitting

5.2.1.4 ResNet50



รูปที่ 5.16 ผลความแม่นยำและค่าความสูญเสียของโมเดลระหว่างการฝึก และ confusion matrix ของโมเดล ResNet50 ภาพขาวเทา

จากกราฟในรูป 5.12 (โดยเส้นกราฟสีน้ำเงินคือการ Train และเส้นกราฟสีส้มคือการ Validation) การฝึกของโมเดล ResNet50 ภาพสีแสดงให้เห็นว่าโมเดลสามารถเข้าใจข้อมูลฝึกได้ดี และข้อมูลทดสอบเองก็ทำได้ดีมากเช่นเดียวกัน

5.3 สรุปผลจากการฝึกฝนโมเดล CNN จากกราฟ

จากกราฟแสดงให้เห็นว่าโมเดลสามารถเรียนรู้ข้อมูลฝึกได้ดี โดยความแม่นยำในการฝึกเพิ่มขึ้นเรื่อย ๆ และค่า Loss ลดลงอย่างชัดเจน อย่างไรก็ตาม ในข้อมูลทดสอบ ความแม่นยำมีความผันผวนและค่าความสูญเสียไม่ลดลงอย่างต่อเนื่อง ซึ่งอาจบ่งบอกถึงการเกิด overfitting ที่โมเดลทำงานได้ดีกับข้อมูลฝึก แต่ความแม่นยำที่ลดลงเมื่อทดสอบกับข้อมูลใหม่

จากการสังเกตในการฝึกสอนโมเดลพบว่า โมเดลมีประสิทธิภาพที่ดีกว่าเมื่อใช้ ภาพสี ซึ่งอาจบ่งบอกว่าโมเดลสามารถจดจำ และแยกแยะลักษณะเด่นจากองค์ประกอบของสีในภาพได้อย่างมีประสิทธิภาพมากกว่าแยกแยะจากลวดลายปูนปั้นโดยตรง สิ่งนี้อาจเป็นผลมาจากการที่โมเดลอาศัยสีเป็นตัวช่วยสำคัญในการจดจำมากกว่าการตรวจจับรายละเอียดเชิงโครงสร้างของลวดลาย

5.4 ตารางสรุปผล

ตารางที่ 5.1 สรุปผลการทดลอง

คลาส	5x5 ภาพสี	11x11 ภาพสี	15x15 ภาพสี	ResNet50 ภาพสี	5x5 ภาพขาวเทา	11x11 ภาพขาวเทา	15x15 ภาพขาวเทา	ResNet50 ภาพขาวเทา
สุโขทัย	84.25%	85.32%	87.45%	99.79%	68.09%	69.36%	67.23%	99.57%
อยุธยาตอนต้น	89.28%	92.40%	89.47%	99.61%	74.07%	87.13%	87.33%	99.22%
อยุธยาตอนกลาง	88.77%	94.18%	93.56%	99.79%	77.13%	94.18%	84.62%	99.58%
อยุธยาตอนปลาย	89.02%	95.70%	93.68%	99.64%	81.03%	89.74%	93.44%	99.64%
รัตนโกสินทร์	86.88%	90.26%	93.04%	99.80%	80.32%	85.49%	88.27%	99.60%
อื่น ๆ	97.50%	95.62%	97.81%	99.69%	94.21%	91.55%	90.45%	100.00%

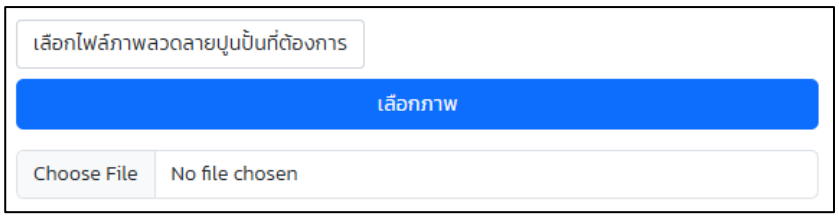
บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ

ในบทนี้จะกล่าวถึงผลการดำเนินงานในเรื่องการทดลองเพื่อหาโมเดลหรืออัลกอริทึมที่จะนำมาใช้ในปริญญานิพนธ์ฉบับนี้ การแสดงความคิดเห็นการวิเคราะห์ผลลัพธ์ที่ได้รวมถึงข้อจำกัดของระบบ และ ข้อเสนอแนะเพื่อนำไป ปรับปรุงแก้ไขให้ดีขึ้น

6.1 ผลการดำเนินงาน

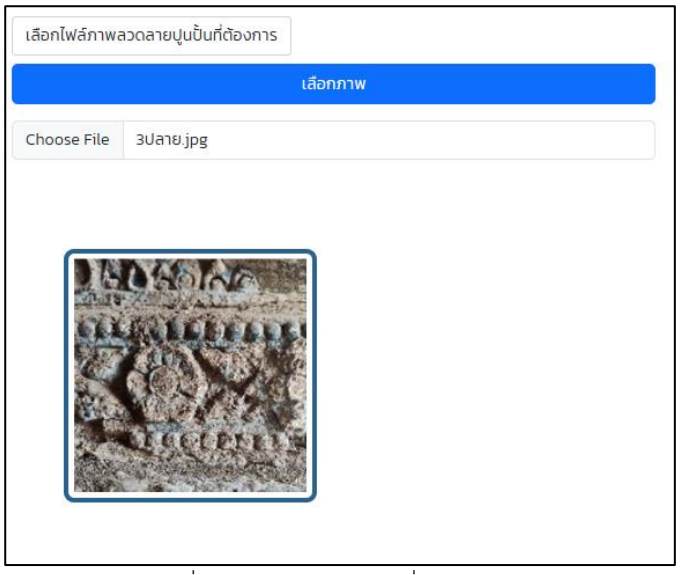
6.1.1 ระบบในเว็บแอพลิเคชั่น

- ระบบอัปโหลดไฟล์ภาพ



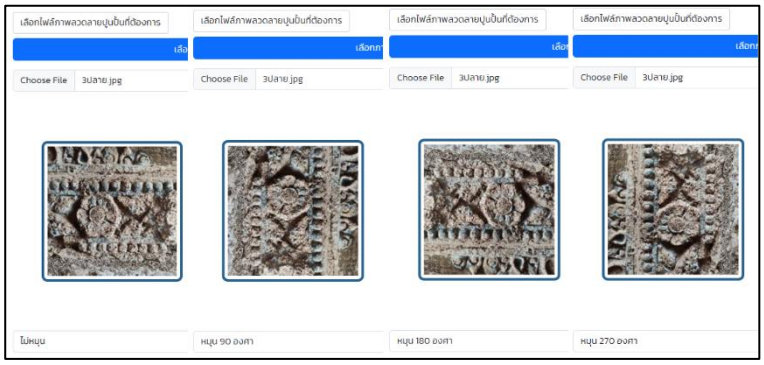
รูปที่ 6.1 ระบบเลือกไฟล์ภาพ

- ระบบแสดงรูปภาพที่ผู้ใช้อัปโหลดก่อนนำไปทำนาย



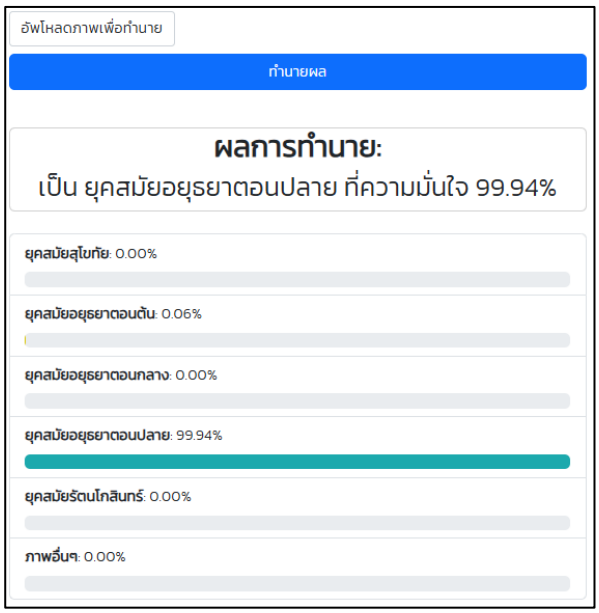
รูปที่ 6.2 ระบบแสดงภาพที่ผู้ใช้อัปโหลด

- ระบบหมุนภาพที่ผู้ใช้อัปโหลด



รูปที่ 6.3 ระบบหมุนภาพที่ผู้ใช้อัปโหลด

- ระบบแสดงความมั่นใจของโมเดลต่อภาพที่ทำนาย



รูปที่ 6.4 ระบบแสดงความมั่นใจของโมเดลต่อภาพที่ทำนาย

6.1.2 การทดลองโมเดล

ในการทดลองของงานวิจัยนี้ ผู้วิจัยยังทำการแบ่งดาต้าเซ็ทออกเป็น 2 แบบได้แก่ ดาต้าเซ็ทภาพสี และดาต้าเซ็ทภาพขาวเทา ในการทดลองจะเปลี่ยนพารามิเตอร์ 1 ตัวคือ ขนาดเคอร์เนล (kernel) โดยขนาดที่ใช้ในการทดลอง มีขนาด5x5, 11x11 และ15x15 และกำหนดค่าแต่ละการทดลองเป็น inputshape=(224,224) ทำการฝึกฝนโมเดล 25 รอบ และค่าอัตราการเรียนรู้ 0.0001 และมีการใช้โมเดล ResNet50 มาเปรียบเทียบกับโมเดลที่สร้างขึ้นมาด้วย

การทดลอง

1. **การทดลองที่ kernel ขนาด 5x5 ดาต้าเซ็ทภาพสี และขาวเทา**

   ในการทดลองนี้พบว่าค่าความแม่นยำเฉลี่ยโดยรวมของ:

   - ดาต้าเซ็ทภาพสีอยู่ที่ 89.28% ผลการทดลองแยกแต่ละคลาสแสดงอยู่ในตารางที่ 6.1

- ดาต้าเซ็ทภาพขาวเทาอยู่ที่ 79.28% ผลการทดลองแยกแต่ละคลาสแสดงอยู่ในตารางที่ 6.2

ในการทดลองนี้ โมเดลที่ใช้ภาพสีมีระดับความแม่นยำที่ค่อนข้างดี ในขณะที่โมเดลที่ใช้ภาพขาวเทามีความแม่นยำลดลงอย่างเห็นได้ชัดเมื่อเปรียบเทียบกัน โดยเฉพาะในกรณีของคลาสสุโขทัยซึ่งมีความแม่นยำที่ต่ำกว่าอย่างชัดเจน

ตารางที่ 6.1 ผลการทดลองโมเดล CNN kernel 5x5 ภาพสี

คลาส	ภาพที่ทำนายถูกต้อง	ภาพที่ทำนายผิด	ความแม่นยำคิดเป็นร้อยละ
สุโขทัย 470 ภาพ	396	74	84.25%
อยุธยาตอนต้น 513 ภาพ	458	55	89.27%
อยุธยาตอนกลาง 481 ภาพ	427	54	88.77%
อยุธยาตอนปลาย 838 ภาพ	746	92	89.02%
รัตนโกสินทร์ 503 ภาพ	437	66	86.87%
ภาพอื่น ๆ 639 ภาพ	623	16	97.49%

ตารางที่ 6.2 ผลการทดลองโมเดล CNN kernel 5x5 ภาพขาวเทา

คลาส	ภาพที่ทำนายถูกต้อง	ภาพที่ทำนายผิด	ความแม่นยำคิดเป็นร้อยละ
สุโขทัย 470 ภาพ	320	150	68.09%
อยุธยาตอนต้น 513 ภาพ	380	133	74.07%
อยุธยาตอนกลาง 481 ภาพ	371	110	77.13%
อยุธยาตอนปลาย 838 ภาพ	679	159	81.03%
รัตนโกสินทร์ 503 ภาพ	404	99	80.32%
ภาพอื่น ๆ 639 ภาพ	602	37	94.21%

2. **การทดลองที่ kernel ขนาด 11x11 ดาต้าเซ็ทภาพสี และขาวเทา**

ในการทดลองนี้พบว่าค่าความแม่นยำเฉลี่ยโดยรวมของ:

- ดาต้าเซ็ทภาพสีอยู่ที่ 92.25% ผลการทดลองแยกแต่ละคลาสแสดงอยู่ในตารางที่ 6.3
- ดาต้าเซ็ทภาพขาวเทาอยู่ 86.24% ผลการทดลองแยกแต่ละคลาสแสดงอยู่ในตารางที่ 6.4

ในการทดลองนี้ โมเดลที่ใช้ภาพสีมีความแม่นยำที่ดีขึ้นอย่างมากเมื่อเทียบกับการทดลองก่อนหน้า ขณะที่โมเดลที่ใช้ภาพขาวเทามีระดับความแม่นยำที่ดีขึ้นโดยรวม แม้ว่าจะยังไม่สามารถเทียบเท่าความแม่นยำของโมเดลภาพสีได้

ตารางที่ 6.3 ผลการทดลองโมเดล CNN kernel 11x11 ภาพสี

คลาส	ภาพที่ทำนายถูกต้อง	ภาพที่ทำนายผิด	ความแม่นยำคิดเป็นร้อยละ
สุโขทัย 470 ภาพ	401	69	85.32%
อยุธยาตอนต้น 513 ภาพ	474	39	92.40%
อยุธยาตอนกลาง 481 ภาพ	453	28	94.18%
อยุธยาตอนปลาย 838 ภาพ	802	36	95.70%
รัตนโกสินทร์ 503 ภาพ	454	49	90.26%
ภาพอื่น ๆ 639 ภาพ	611	28	95.62%

ตารางที่ 6.4 ผลการทดลองโมเดล CNN kernel 11x11 ภาพขาวเทา

คลาส	ภาพที่ทำนายถูกต้อง	ภาพที่ทำนายผิด	ความแม่นยำคิดเป็นร้อยละ
สุโขทัย 470 ภาพ	326	144	69.36%
อยุธยาตอนต้น 513 ภาพ	447	66	87.13%
อยุธยาตอนกลาง 481 ภาพ	453	28	94.18%
อยุธยาตอนปลาย 838 ภาพ	752	86	89.74%
รัตนโกสินทร์ 503 ภาพ	430	73	85.49%
ภาพอื่น ๆ 639 ภาพ	585	54	91.55%

3. **การทดลองที่ kernel ขนาด 15x15 ดาต้าเซ็ทภาพสี และขาวเทา**

ในการทดลองนี้พบว่าค่าความแม่นยำเฉลี่ยโดยรวมของ:

- ดาต้าเซ็ทภาพสีอยู่ที่ 92.50% ผลการทดลองแยกแต่ละคลาสแสดงอยู่ในตารางที่ 6.5
- ดาต้าเซ็ทภาพขาวเทาอยู่ที่ 85.22% ผลการทดลองแยกแต่ละคลาสแสดงอยู่ในตารางที่ 6.6

ในการทดลองนี้ โมเดลที่ใช้ภาพสีมีระดับความแม่นยำที่สูงขึ้นไปอีกขั้นเมื่อเทียบกับการทดลองครั้งก่อน ในขณะที่โมเดลที่ใช้ภาพขาวเทากลับมีระดับความแม่นยำลดลงเมื่อเทียบกับการใช้ kernel ขนาด 11x11

ตารางที่ 6.5 ผลการทดลองโมเดล CNN kernel 15x15 ภาพสี

คลาส	ภาพที่ทำนายถูกต้อง	ภาพที่ทำนายผิด	ความแม่นยำคิดเป็นร้อยละ
สุโขทัย 470 ภาพ	411	59	87.45%
อยุธยาตอนต้น 513 ภาพ	459	54	89.47%
อยุธยาตอนกลาง 481 ภาพ	450	31	93.56%
อยุธยาตอนปลาย 838 ภาพ	785	53	93.68%
รัตนโกสินทร์ 503 ภาพ	468	35	93.04%
ภาพอื่น ๆ 639 ภาพ	625	14	97.81%

ตารางที่ 6.6 ผลการทดลองโมเดล CNN kernel 15x15 ภาพขาวเทา

คลาส	ภาพที่ทำนายถูกต้อง	ภาพที่ทำนายผิด	ความแม่นยำคิดเป็นร้อยละ
สุโขทัย 470 ภาพ	316	154	67.23%
อยุธยาตอนต้น 513 ภาพ	448	65	87.33%
อยุธยาตอนกลาง 481 ภาพ	407	74	84.62%
อยุธยาตอนปลาย 838 ภาพ	783	55	93.44%
รัตนโกสินทร์ 503 ภาพ	444	59	88.27%
ภาพอื่น ๆ 639 ภาพ	578	61	90.45%

4. การทดลองที่ใช้โมเดล ResNet50 ดาต้าเซ็ทภาพสี และขาวเทา

ในการทดลองนี้พบว่าค่าความแม่นยำเฉลี่ยโดยรวมของ:

- ดาต้าเซ็ทภาพสีอยู่ที่ 99.72% ผลการทดลองแยกแต่ละคลาสแสดงอยู่ในตารางที่ 6.7
- ดาต้าเซ็ทภาพขาวเทาอยู่ที่ 99.60% ผลการทดลองแยกแต่ละคลาสแสดงอยู่ในตารางที่ 6.8

ในการทดลองนี้ โมเดล ResNet50 ทั้งภาพสีและภาพขาวเทามีความแม่นยำโดยรวมดีมาก

ตารางที่ 6.7 ผลการทดลองโมเดล ResNet50 ภาพสี

คลาส	ภาพที่ทำนายถูกต้อง	ภาพที่ทำนายผิด	ความแม่นยำคิดเป็นร้อยละ
สุโขทัย 470 ภาพ	469	1	99.79%
อยุธยาตอนต้น 513 ภาพ	511	2	99.61%
อยุธยาตอนกลาง 481 ภาพ	480	1	99.79%
อยุธยาตอนปลาย 838 ภาพ	835	3	99.64%
รัตนโกสินทร์ 503 ภาพ	502	1	99.80%
ภาพอื่น ๆ 639 ภาพ	637	2	99.69%

ตารางที่ 6.8 ผลการทดลองโมเดล ResNet50 ภาพขาวเทา

คลาส	ภาพที่ทำนายถูกต้อง	ภาพที่ทำนายผิด	ความแม่นยำคิดเป็นร้อยละ
สุโขทัย 470 ภาพ	468	2	99.57%
อยุธยาตอนต้น 513 ภาพ	509	4	99.22%
อยุธยาตอนกลาง 481 ภาพ	479	2	99.58%
อยุธยาตอนปลาย 838 ภาพ	835	3	99.64%
รัตนโกสินทร์ 503 ภาพ	501	2	99.60%
ภาพอื่น ๆ 639 ภาพ	639	0	100.00%

สรุปผลการทดลองของโมเดล CNN

ความแม่นยำเมื่อใช้ใช้ kernel ขนาดใหญ่ขึ้น

- **CNN ภาพสี**

ตารางที่ 6.9 สรุปความแม่นยำสูงสุดแต่ละคลาสของโมเดลภาพสี

คลาส	**ความแม่นยำสูงสุด**	**ขนาด kernel ที่มีความแม่นยำสูงสุด**
สุโขทัย	87.45%	15x15
อยุธยาตอนต้น	92.40%	11x11
อยุธยาตอนกลาง	94.18%	11x11
อยุธยาตอนปลาย	95.70%	11x11
รัตนโกสินทร์	93.04%	15x15
อื่น ๆ	97.81%	15x15

- **CNN ขาวเทา**

ตารางที่ 6.10 สรุปความแม่นยำสูงสุดแต่ละคลาสของโมเดลภาพสี

คลาส	**ความแม่นยำสูงสุด**	**ขนาด kernel ที่มีความแม่นยำสูงสุด**
สุโขทัย	69.36%	11x11
อยุธยาตอนต้น	87.33%	15x15
อยุธยาตอนกลาง	94.18%	11x11
อยุธยาตอนปลาย	93.44%	15x15
รัตนโกสินทร์	88.27%	15x15
อื่น ๆ	94.21%	5x5

ข้อสรุปของการทดลองเพิ่มขนาด kernel ในโมเดล CNN

- ภาพสีความแม่นยำสูงกว่าในทุกคลาส โดยเฉพาะ kernel ขนาดใหญ่ (11x11 และ 15x15) ช่วยให้จำแนกได้ดีขึ้น
- ภาพขาวเทาความแม่นยำต่ำกว่าโดยเฉพาะในคลาสสุโขทัย แต่ยังคงมีประสิทธิภาพดีเมื่อใช้ kernel ขนาด 11x11 หรือ 15x15 ในบางคลาส
- โดยรวมภาพสีและ kernel ขนาดใหญ่เป็นตัวเลือกที่เหมาะสม
- จากการสังเกตในการฝึกสอนโมเดลพบว่า โมเดลมีประสิทธิภาพที่ดีกว่าเมื่อใช้ ภาพสี ซึ่งอาจบ่งบอกว่าโมเดลสามารถจดจำ และแยกแยะลักษณะเด่นจากองค์ประกอบของสีในภาพได้อย่างมีประสิทธิภาพมากกว่าแยกแยะจากลวดลายปูนปั้นโดยตรง สิ่งนี้อาจเป็นผลมาจากการที่โมเดลอาศัยสีเป็นตัวช่วยสำคัญในการจดจำมากกว่าการตรวจจับรายละเอียดเชิงโครงสร้างของลวดลาย
- จากการทดลองข้างต้น ผู้วิจัยจึงสรุปว่าโมเดล CNN ที่มีประสิทธิภาพดีที่สุดคือโมเดลภาพสีที่ขนาด kernel 15x15 และผลการทดลองสรุปดังแสดงในตารางที่ 6.3

สรุปผลการเปรียบเทียบระหว่าง CNN และ ResNet50

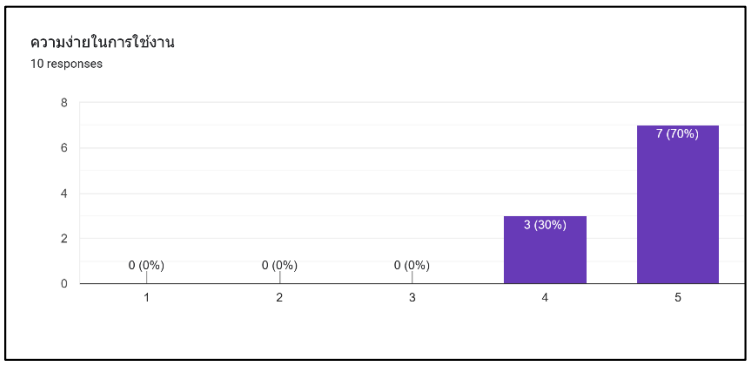
โมเดล ResNet50 แสดงความแม่นยำที่สูงขึ้นอย่างชัดเจนเมื่อเทียบกับโมเดล CNN โดยทั้งภาพสีและภาพขาวเทามีความแม่นยำใกล้เคียงกัน โดย ResNet50 สามารถทำความแม่นยำได้สูงมากในทั้งสองรูปแบบ (มากกว่า 99% ในทุกคลาส) ซึ่งดีกว่าโมเดล CNN อย่างมาก ในทางกลับกัน โมเดล CNN มีความแม่นยำที่ต่ำกว่า โดยเฉพาะในกรณีที่ใช้ภาพขาวเทาร่วมกับ kernel ขนาดเล็ก ข้อแตกต่างหลักระหว่างโมเดลทั้งสองคือ ResNet50 มีจำนวนเลเยอร์มากกว่าและมี shortcut connections ซึ่งช่วยแก้ปัญหาการสูญเสียข้อมูลเชิงลึกในระหว่างการเรียนรู้ ขณะที่ CNN ไม่มีโครงสร้างนี้ ส่งผลให้ประสิทธิภาพโดยรวมของ CNN ลดลง โดยเฉพาะเมื่อใช้กับภาพขาวเทาและ kernel ขนาดเล็ก

6.2 ความพึงพอใจผู้ใช้งาน

จากการดำเนินโครงงานการจำแนกยุคสมัยประติมากรรมลวดลายปูนปั้นประดับโบราณสถานไทยด้วยการเรียนรู้เชิงลึก ได้ทำการสำรวจจากกลุ่มตัวอย่างจำนวน 10 คน ที่มีความสนใจในด้านประติมากรรมลวดลายปูนปั้นอดีต โดยที่ผู้เข้าร่วมการทดสอบจะมีการให้คะแนนการประเมินความพึงพอใจของเว็บแอพพลิเคชัน โดยเลือกตั้งแต่ 1 ดาว (ความพึงพอใจน้อย) ไปจนถึง 5 ดาว (ความพึงพอใจมาก) โดยประเมินในด้านความสะดวกในการใช้งาน และด้านการออกแบบหน้าเว็บ

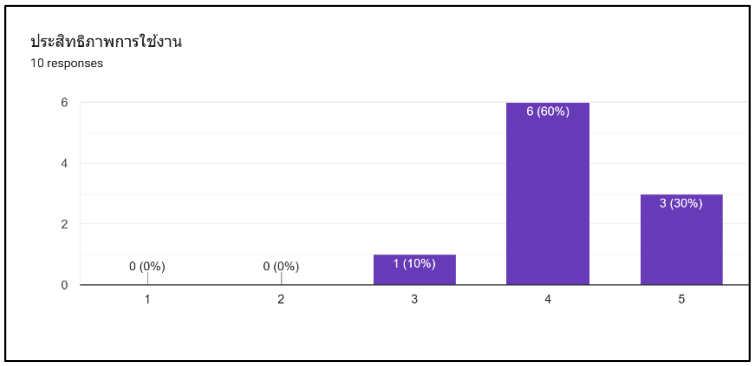
6.2.1 ด้านความสะดวกในการใช้

- ความง่ายในการใช้งาน



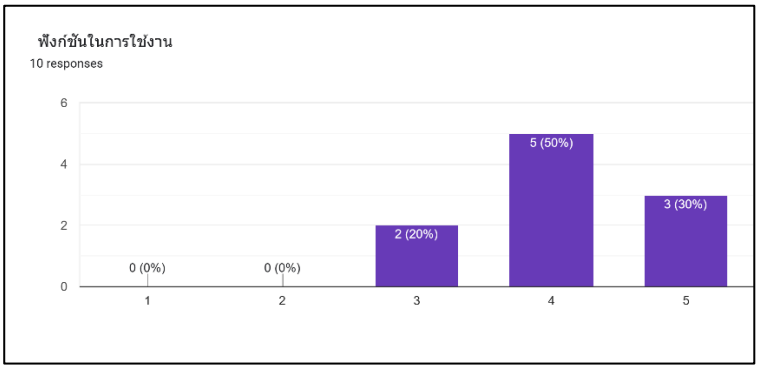
รูปที่ 6.5 ผลการประเมินความง่ายในการใช้งาน

- ประสิทธิภาพการใช้งาน



รูปที่ 6.6 ผลการประเมินประสิทธิภาพการใช้งาน

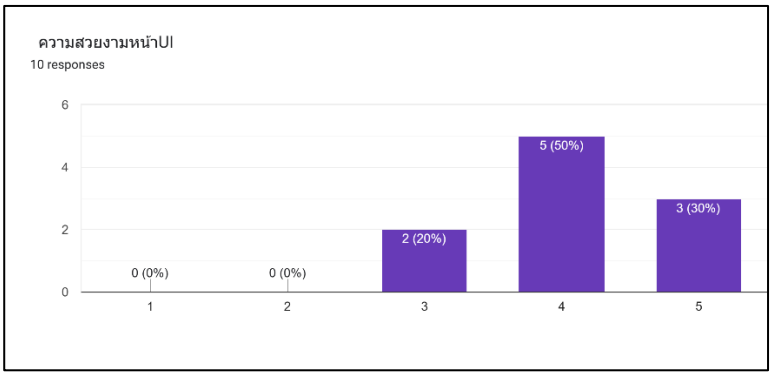
- ฟังก์ชันในการใช้งาน



รูปที่ 6.7 ผลการประเมินฟังก์ชันการใช้งาน

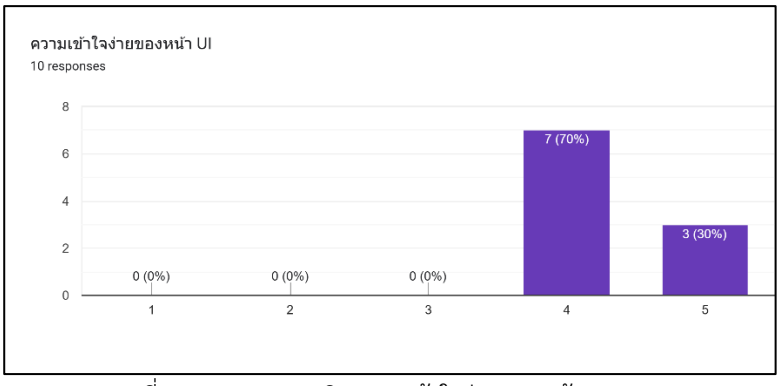
6.2.2 ด้านการออกแบบหน้าเว็บ

- ความสวยงามหน้า UI



รูปที่ 6.8 ผลการประเมินความสวยงามหน้า UI

- ความเข้าใจง่ายของหน้า UI



รูปที่ 6.9 ผลการประเมินความเข้าใจง่ายของหน้า UI

6.2.3 สรุปความพึงพอใจผู้ใช้

เกณฑ์การประเมินมีคะแนนกำหนด ดังนี้

- ความพึงพอใจระดับดีมาก 4.00-5.00
- ความพึงพอใจระดับดี 3.00-4.00
- ความพึงพอใจระดับปานกลาง 2.00-3.00
- ความพึงพอใจระดับพอใช้ 1.00-2.00
- ความพึงพอใจระดับน้อย 0.00-1.00

ตารางที่ 6.11 สรุปความพึงพอใจผู้ใช้

ความพึงพอใจ	**คะแนนเฉลี่ย**	**ระดับการประเมิน**
ด้านความสะดวกในการใช้		
ความง่ายในการใช้งาน	4.70	ดีมาก
ประสิทธิภาพการใช้งาน	4.20	ดีมาก
ฟังก์ชันการใช้งาน	4.10	ดีมาก
ด้านการออกแบบหน้าเว็บ		
ความสวยงามหน้า UI	4.10	ดีมาก
ความเข้าใจง่ายของหน้า UI	4.30	ดีมาก

6.3 ข้อจำกัด

1. การอัปโหลดภาพต้องเห็นลักษณะต่าง ๆ ของลวดลายหากเห็นส่วนใดส่วนหนึ่งหรือ เห็นลักษณะที่ไม่ชัดเจนจะมีผลต่อการทำนาย
2. ผู้ใช้งานทำนายภาพได้ครั้งละ 1 ภาพ ไม่สามารถทำนายครั้งละหลาย ๆ ภาพในครั้งเดียวได้
3. ผู้ใช้งานไม่สามารถเปลี่ยนโมเดลเพื่อการทำนายได้
4. หากไฟล์รูปภาพมีความละเอียดน้อยกว่าที่ขนาดขั้นต่ำแนะนำมากเกินไปจะทำให้รูปที่ใช้ทำนายมีภาพที่ไม่ชัดเจนโมเดลอาจหาฟีเจอร์ของรูปได้ยากจะทำให้ผลลัพธ์การทำนายผิดพลาดได้
5. เว็บแอปพลิเคชันสามารถจำแนกได้แค่ยุคสมัยเท่านั้นไม่สามารถจำแนกลวดลายได้ หากเป็นภาพอื่นนอกจากภาพปูนปั้นแล้ว จะถูกจัดเป็นภาพอื่น ๆ
6. เนื่องจากภาพที่ใช้ฝึกโมเดลตอนนี้ มีความชัดที่ต่ำ จึงทำให้การทำ edge detection เป็นไปได้ไม่ดีเท่าที่ควร จึงทำการตัดออกจากการทดลอง

6.4 ข้อเสนอแนะ

1. ทำให้โมเดลสามารถจำแนกลวดลายพร้อมกับยุคสมัยได้
2. สามารถเปลี่ยนโมเดลที่ใช้ในการทำนายบนเว็บแอปพลิเคชันได้
3. เพิ่มความละเอียดของภาพ Dataset เพื่อเพิ่มความละเอียดในการฝึกโมเดล และทำ edge detection ได้ดีขึ้น
4. เพิ่มไฟล์ภาพเพื่อทำนายพร้อมกันได้ในหลาย ๆ ภาพ

5. อาจจะเปลี่ยนวิธีเพิ่มวิธีการ Augment ดาต้าเซ็ทด้วยวิธีการ Random crop
6. ทำให้โมเดลสามารถตรวจจับและตีกรอบได้ว่าบริเวณไหนของภาพเป็นลวดลายปูนปั้น
7. ในการขั้นตอนการทำดาต้าเซ็ทอาจะตองทำการลบส่วนที่ไม่ใช่ลวดลายปูนปั้นออกไป

บรรณานุกรม

[1] ด. ส. เล็กสุขุม, ลวดลายแบบอยุธยาตอนต้น, Bangkok: เมืองโบราณ, 2566.

[2] ด. ส. เล็กสุขุม, ลวดลายแบบอยุธยาตอนกลาง, Bangkok: เมืองโบราณ, 2567.

[3] ด. ส. เล็กสุขุม, ลวดลายแบบอยุธยาตอนปลาย, Bangkok: เมืองโบราณ, 2566.

[4] มูลนิธิโครงการสารานุกรมไทยสำหรับเยาวชนฯ, "พัฒนาการของลายไทย-ลายกระหนก โดยลำดับยุคสมัย ตั้งแต่แรกเริ่มจนถึงปัจจุบัน," มูลนิธิโครงการสารานุกรมไทยสำหรับเยาวชนฯ, [ออนไลน์]. Available: https: / / www. saranukromthai. or. th/ sub/ book/ book. php?book= 3 8 &chap= 1 &page= t3 8 - 1 - infodetail02.html. [%1 ที่เข้าถึง16 August 2024].

[5] อ. เรืองสุข, "CLASSIFICATION OF BUDDHA AMULETS USING DEEP CONVOLUTIONAL NEURAL

[6] A. Jain, "Data Augmentation," medium, 16 Feb 2024. [ออนไลน์]. Available: https://medium.com/@abhishekjainindore24/data-augmentation-00c72f5f4c54. [%1 ที่เข้าถึง16 August 2024].

[7] Apple, "PlantSnap - identify plants," PlantSnap Inc, [ออนไลน์]. Available: https://apps.apple.com/au/story/id1345309945. [%1 ที่เข้าถึง12 March 2024].

[8] Kwankamon Dittakan and Nawanol Theera-Ampornpunt, "Pum-Riang Thai Silk Pattern Classification," ใน *PRICAI*, Bangkok, 2018.

[9] N. Phongchit, "Convolutional Neural Network (CNN) คืออะไร," medium, 19 sep 2018. [ออนไลน์]. Available: https://medium.com/@natthawatphongchit/มาลองดูวิธีการคิดของ-cnn-กัน-e3f5d73eebaa. [%1 ที่เข้าถึง16 February 2024].

[10] N. Phongchit, "มาทำความรู้จัก ResNet กันดีกว่า," medium, 18 Mar 2020. [ออนไลน์]. Available: https://medium.com/@natthawatphongchit/มาทำความรู้จัก-resnet-กันดีกว่า-aec3a8c10793. [%1 ที่เข้าถึง 16 August 2024].

[11] NETWORKS," 2564. [ออนไลน์]. Available: https://libdoc.dpu.ac.th/thesis/Attapol.Rou.pdf. [%1 ที่เข้าถึง16 August 2024].

[12] nishant_kumar, "Digital Image Processing Basics," medium, 22 Feb 2023. [ออนไลน์]. Available: https://www.geeksforgeeks.org/digital-image-processing-basics/. [%1 ที่เข้าถึง16 August 2024].

ภาคผนวก ก คู่มือการติดตั้งโปรแกรม

ก.1 วิธีการติดตั้ง Visual studio code

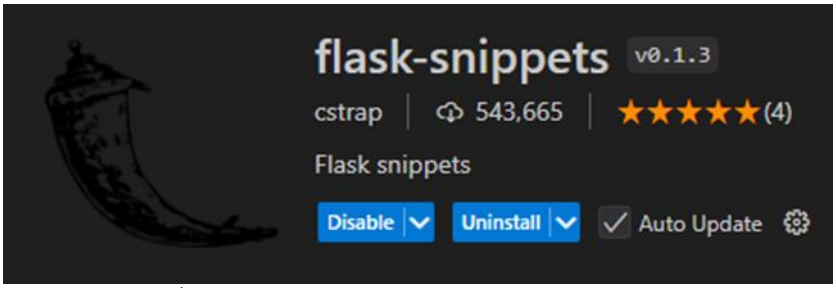
ก.1.1 ดาวน์โหลด Visual studio code ที่เว็บไซต์ code.visualstudio.com/download

รูปผนวกที่ ก.1 เว็บไซต์ code.visualstudio.com/download

ก.1.2 ติดตั้ง Extension Python ใน Visual Studio Code

รูปผนวกที่ ก.2 Extension Python ใน Visual Studio Code

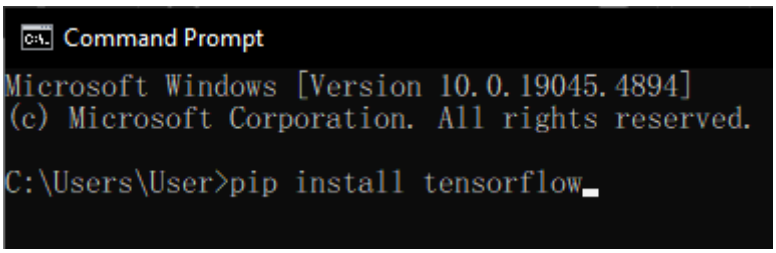
ก.1.3 ติดตั้ง Extension Flask Snippet ใน Visual Studio Code



รูปผนวกที่ ก.3 Extension Flask Snippet ใน Visual Studio Code

ก.2 การติดตั้ง Library ที่จำเป็นต่อ Project

ก.2.1 การติดตั้ง TensorFlow พิมพ์ pip install tensorflow ลงใน cmd



รูปผนวกที่ ก.4 pip install tensorflow ใน cmd

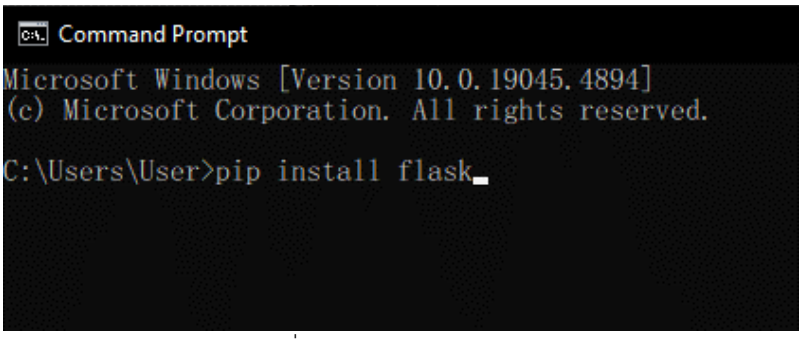
ก.2.2 การติดตั้ง Pillow พิมพ์ pip install Pillow ลงใน cmd

รูปผนวกที่ ก.5 pip install Pillow ใน cmd

ก.2.3 วิธีการติดตั้ง numpy พิมพ์ pip install numpy ลงใน cmd

รูปผนวกที่ ก.6 pip install numpy ใน cmd

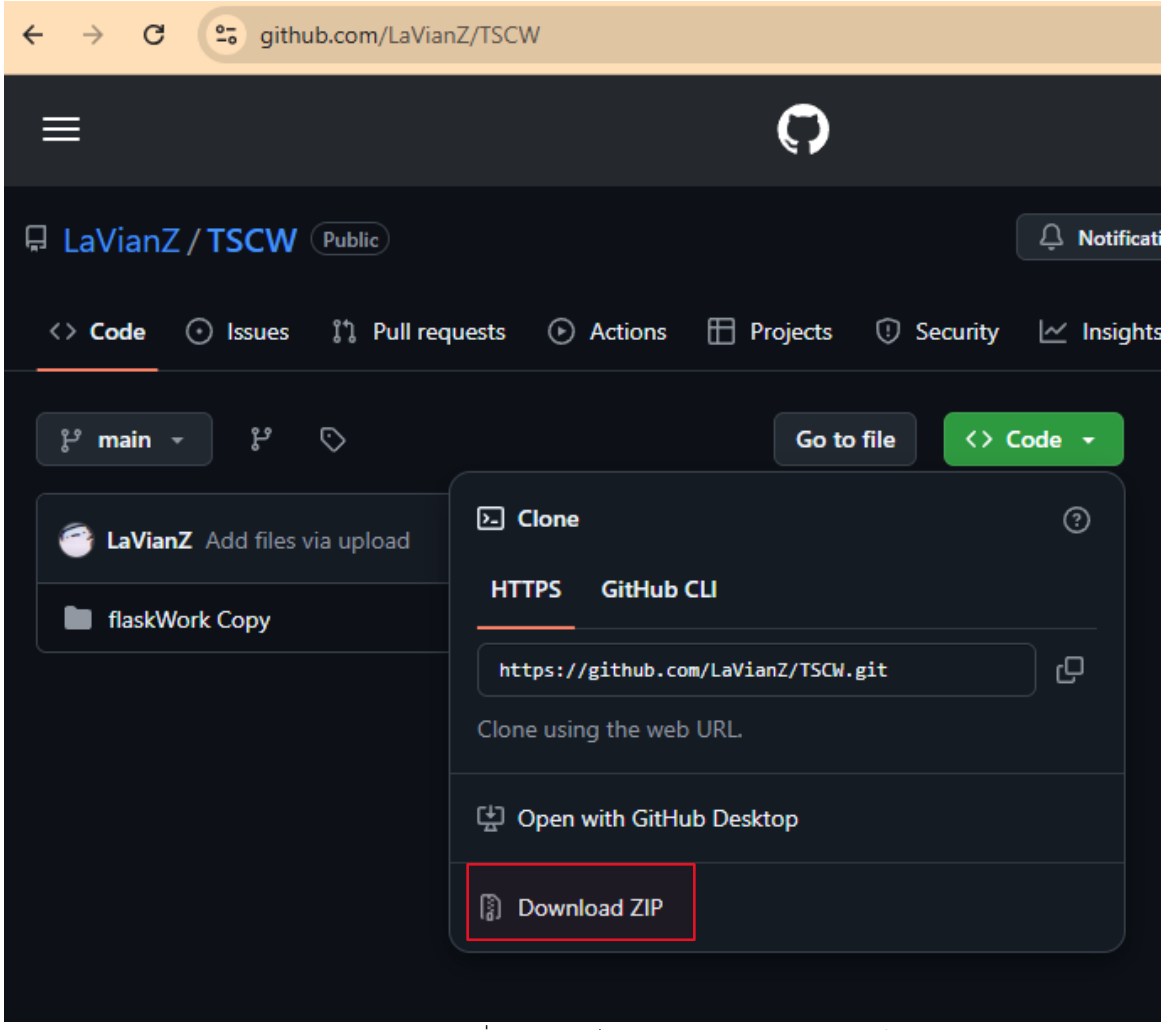
ก.2.4 วิธีการติดตั้ง Flask พิมพ์ pip install flask ลงใน cmd



รูปผนวกที่ ก.7 pip install flask ใน cmd

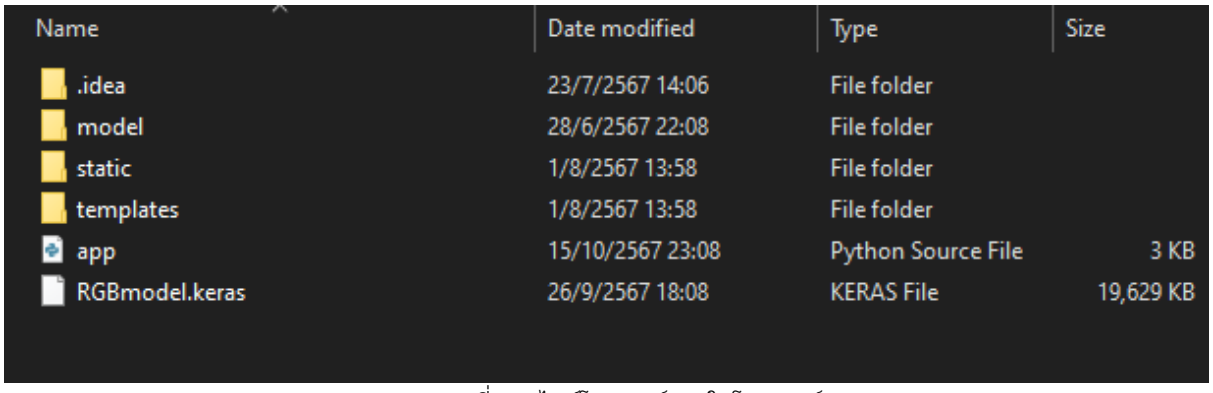
ก.3 การติดตั้งโปรเจกต์

การติดตั้งโปรเจกต์นั้นให้เข้าไปที่เว็บไซต์ https://github.com/LaVianZ/TSCW เพื่อทำการดาวน์โหลดไฟล์โปรเจกต์



รูปผนวกที่ ก.8 หน้าเว็บ Github สำหรับดาวน์โหลดไฟล์โปรเจกต์

จากนั้นทำการแตกไฟล์โปรเจกต์ จะได้ไฟล์ดังภาพ



Name	Date modified	Type	Size
.idea	23/7/2567 14:06	File folder	
model	28/6/2567 22:08	File folder	
static	1/8/2567 13:58	File folder	
templates	1/8/2567 13:58	File folder	
app	15/10/2567 23:08	Python Source File	3 KB
RGBmodel.keras	26/9/2567 18:08	KERAS File	19,629 KB

รูปผนวกที่ ก.9 ไฟล์โปรเจกต์ภายในโฟลเดอร์

การรันไฟล์โปรเจกต์นั้น ให้ทำการกดรันไฟล์ในโฟลเดอร์โปรเจกต์ที่มีชื่อว่า app.py เพื่อเปิดระบบหลังบ้าน

```
26      image = preprocess_image(image)
27
28      raw_prediction = model.predict(image)[0]
29      print("Raw prediction:", raw_prediction)
30      # Apply softmax to get probabilities
31      prediction = tf.nn.softmax(raw_prediction)
32      print("prediction:", prediction)
```

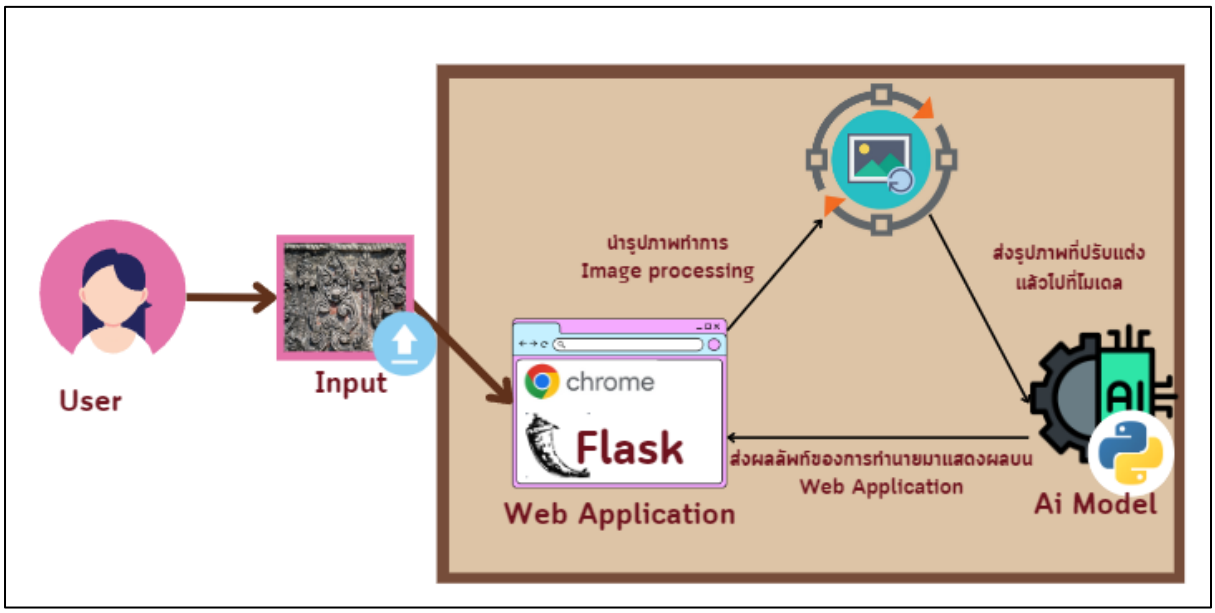
```
PROBLEMS 1   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\User\Desktop\flaskWork Copy> & C:/Users/User/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/User/Desktop/flaskWork Copy/app.py"
2024-10-06 15:25:12.099141: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floa
set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2024-10-06 15:25:12.673283: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floa
set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2024-10-06 15:25:13.838353: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in per
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

รูปผนวกที่ ก.10 รัน Project ที่ app.py

การทำงานของ Server จะใช้ Flask ในแบบจำลอง Server ของระบบบน Windows จะใช้รูปแบบการติดต่อ Domain คือ localhost ซึ่งมีรูปแบบคือ http://127.0.0.1:5000/

ภาคผนวก ข การทำงานของโปรแกรม

ข.1 แผนผังการทำงานของระบบ



รูปผนวกที่ ข.1 แผนผังการทำงานของระบบ

ข.2 ฟังก์ชันในโปรแกรม

ข.2.1 ฟังก์ชันปรับขนาดภาพที่รับเข้ามา

```
def preprocess_image(image):

    image = image.resize((224, 224))
    #image = np.array(image).astype('float32') / 255.0

    # Add batch dimension (1, 224, 224, 3)
    image = np.expand_dims(image, axis=0)
    return image
```

รูปผนวกที่ ข.2 ฟังก์ชันปรับขนาดภาพที่รับเข้ามา

ฟังก์ชัน preprocess_image() นี้ทำการปรับขนาดภาพให้เป็น 224x224 พิกเซลเพื่อให้สอดคล้องกับความต้องการของโมเดล

ข.2.2 ฟังก์ชันทำนาย

```
def predict_image(model, image):

    image = preprocess_image(image)

    raw_prediction = model.predict(image)[0]
    print("Raw prediction:", raw_prediction)

    prediction = tf.nn.softmax(raw_prediction)
    print("prediction:", prediction)

    cat=["ยุคสมัยสุโขทัย","ยุคสมัยอยุธยาตอนต้น","ยุคสมัยอยุธยาตอนกลาง","ยุคสมัยอยุธยาตอนปลาย","ยุคสมัยรัตนโกสินทร์","ภาพอื่นๆ"]

    response = {
        "predicted_result": [
            {"number": cat[i], "probability": float(f"{prediction[i] * 100:.2f}")} for i in range(6)
        ]
    }

    return response
```

รูปผนวกที่ ข.3 ฟังก์ชันทำนาย

ฟังก์ชัน predict_image() ทำการรับภาพที่ได้รับมาจากฟังก์ชัน preprocess_image() จากนั้นส่งภาพเข้าสู่โมเดลเพื่อทำการทำนายโดยโมเดลที่ใช้ เป็นโมเดลCNN kernel 5x5 ผลลัพธ์ที่ได้จะใช้ softmax เพื่อแปลงเป็นความน่าจะเป็นสำหรับแต่ละคลาส จากนั้นจะจัดเก็บผลลัพธ์ในรูปแบบ JSON

ข.2.3 ฟังก์ชันตรวจสอบไฟล์ และผลลัพธ์

```
@app.route('/predict', methods=['POST'])
def predict():
    if 'file' not in request.files:
        return jsonify({"error": "No file part"}), 400

    file = request.files['file']

    if file.filename == '':
        return jsonify({"error": "No selected file"}), 400

    if file:
        # Read the image
        image = Image.open(io.BytesIO(file.read()))

        # Predict the image
        response = predict_image(model, image)

        return jsonify(response)
```

รูปผนวกที่ ข.4 ฟังก์ชันตรวจสอบไฟล์ และผลลัพธ์

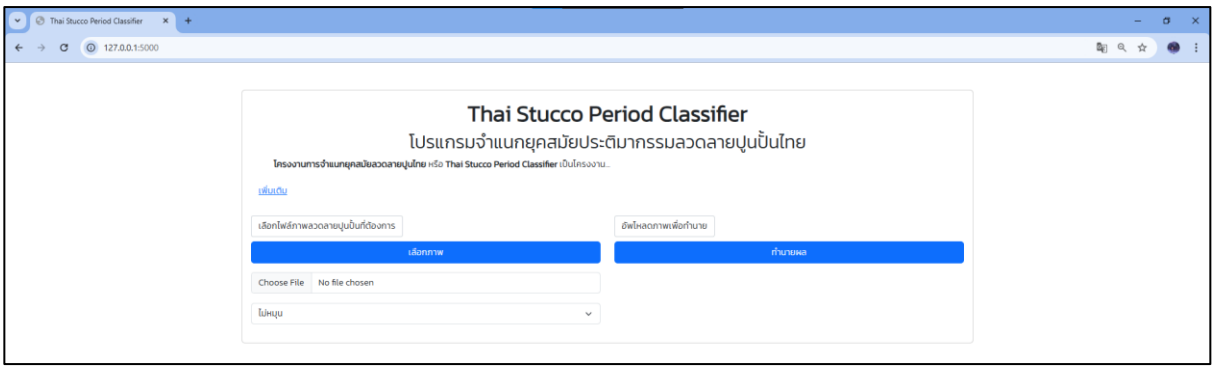
ส่วนนี้นี้เป็นการสร้าง endpoint /predict ใน Flask ที่รับคำขอแบบ POST เพื่อทำนายภาพที่ส่งเข้ามา หากไม่มีไฟล์แนบหรือไฟล์ไม่ได้ถูกเลือก ระบบจะส่งข้อความแสดงข้อผิดพลาด แต่หากมีไฟล์ระบบจะอ่านไฟล์ภาพนั้นด้วย Image.open และส่งภาพไปยังฟังก์ชัน predict_image() เพื่อประมวลผลและส่งผลลัพธ์กลับมาในรูปแบบ JSON

ภาคผนวก ค คู่มือการใช้โปรแกรม

ค.1 หน้าหลัก

เมื่อเปิดโปรแกรมขึ้นมาจะพบกับหน้าหลัก

- ชื่อโปรแกรม Thai Stucco Period Classifier แสดงอยู่ด้านบนสุดพร้อมคำอธิบายเป็นภาษาไทย
- ลิงก์ "เพิ่มเติม" ลิงก์นี้อยู่ใต้คำอธิบายโครงการ ใช้เพื่อแสดงรายละเอียดเพิ่มเติมเกี่ยวกับโครงงานและลวดลายปูนปั้น
- มีข้อความแนะนำ "เลือกไฟล์ภาพลวดลายปูนปั้นที่ต้องการ" และปุ่ม "เลือกภาพ"
- ปุ่ม "ทำนายผล" เพื่อสั่งให้ระบบเริ่มทำการทำนายยุคสมัยของลายปูนปั้นที่อัปโหลดมา
- ดรอปดาวน์การหมุน ใช้สำหรับหมุนภาพที่อัปโหลด โดยมีตัวเลือกให้ผู้ใช้หมุนภาพเป็น 90, 180, และ 270 องศา

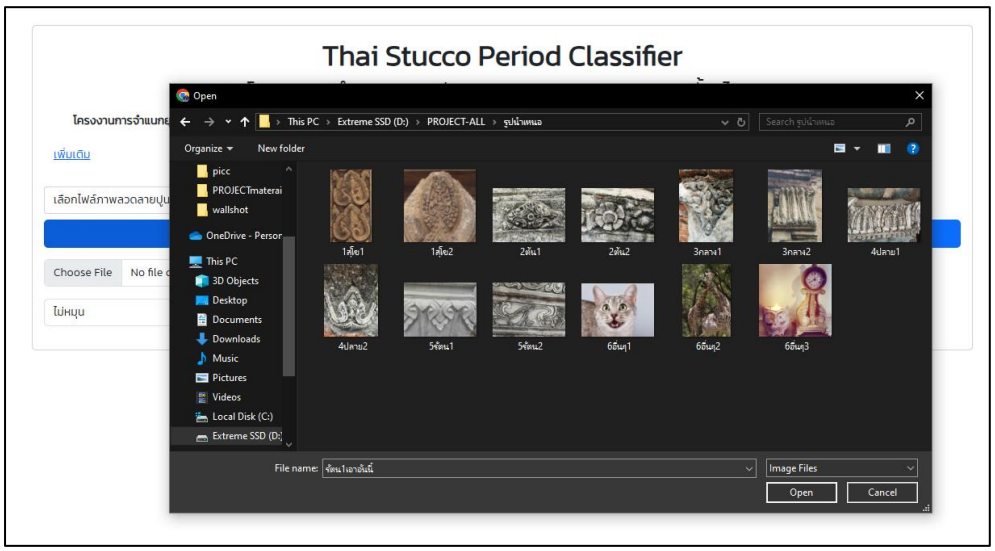


รูปผนวกที่ ค.1 หน้าหลัก

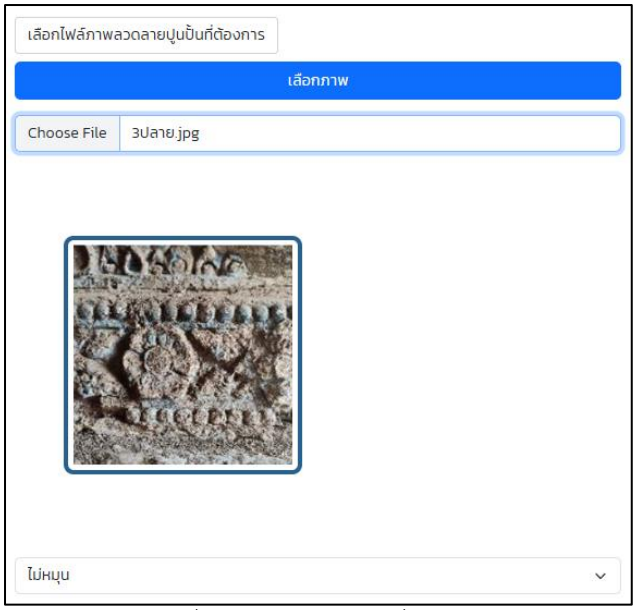
ค.2 ขั้นตอนการใช้งาน

ค.2.1 การอัปโหลดภาพ

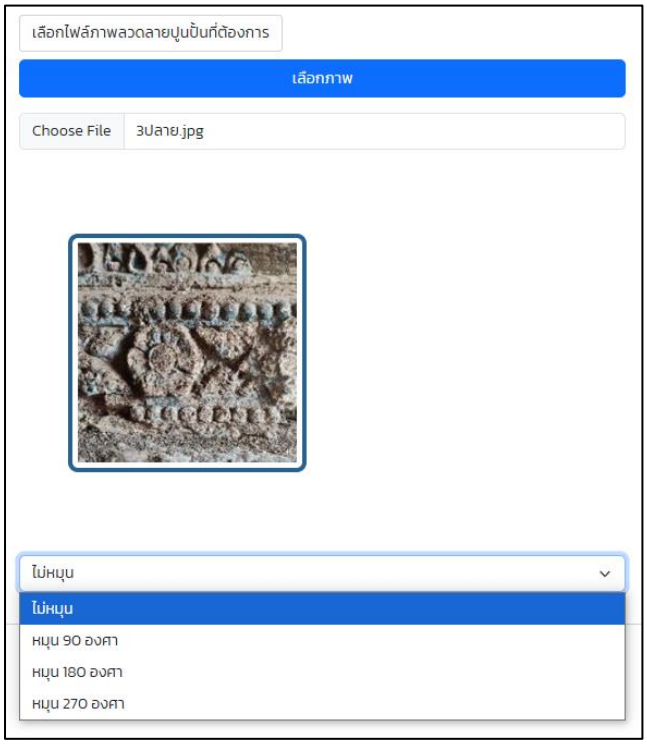
เริ่มจากกดปุ่ม "เลือกภาพ" เพื่อแสดงหน้าต่างหาไฟล์ภาพของผู้ใช้ภายในเครื่องดังรูปผนวกที่ ค.2 และให้ผู้ใช้เลือกภาพที่จะมาทำนายด้วยการดับเบิ้ลคลิก หรือ กดปุ่ม Open หลังจากอัปโหลดภาพแล้ว จะแสดงภาพที่ผู้ใช้เลือกมาดังรูปผนวกที่ ค.3 และผู้ใช้สามารถหมุนภาพได้ดังภาพที่ ค.4



รูปผนวกที่ ค.2 หน้าต่างเลือกภาพ



รูปผนวกที่ ค.3 หน้าแสดงภาพที่ผู้ใช้เลือกมา

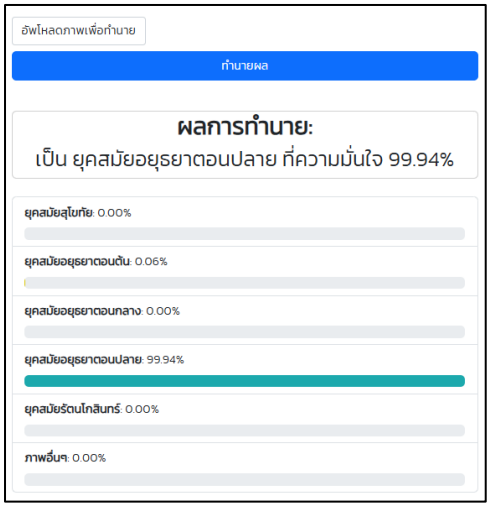


รูปผนวกที่ ค.4 ดรอปดาวน์การหมุน

ค.2.2 การทำนายภาพ

เมื่ออัปโหลดรูปภาพเรียบร้อยแล้วให้ทำการกดปุ่ม "ทำนายผล" โปรแกรมก็จะเริ่มทำการทำนายภาพที่ผู้ใช้เลือก รอผลทำนายสักครู่ หลังจากนั้นก็จะแสดงผลการทำนายออกมาดัง

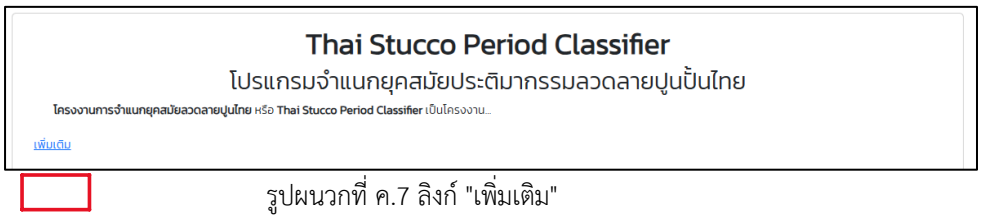
รูปผนวกที่ ค.5 ระหว่างรอทำนายผล



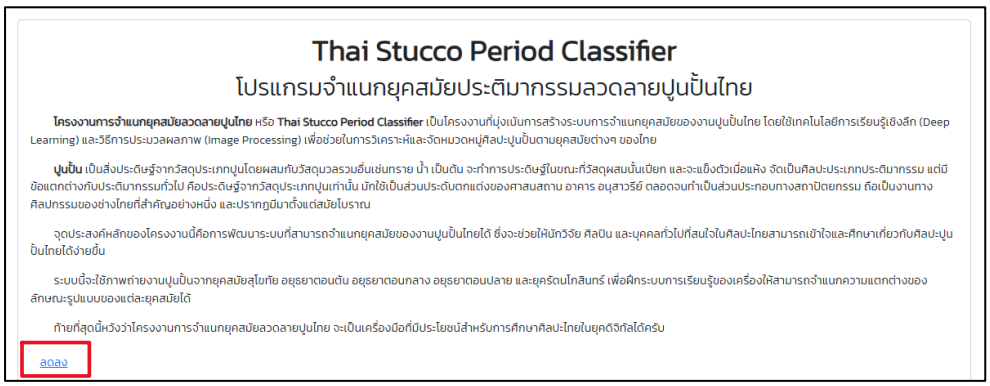
รูปผนวกที่ ค.6 หน้าแสดงผลทำนาย

ค.2.3 การแสดงข้อมูลเพิ่มเติม

หากต้องการอ่านข้อมูลของโครงงานเพิ่มเติมให้กดลิงก์ "เพิ่มเติม" ในรูปผนวกที่ ค.7 เพื่อแสดงข้อความอธิบายโครงงานและลวดลายปูนปั้นที่ซ่อนไว้ ดังรูปผนวกที่ ค.8 และสามารถกดลิงก์ "ลดลง" ที่รูปผนวกที่ ค.8 เพื่อลดข้อความเป็นแบบรูปผนวกที่ ค.7



รูปผนวกที่ ค.7 ลิงก์ "เพิ่มเติม"



รูปผนวกที่ ค.8 ลิงก์ "ลดลง"

ภาคผนวก ง แบบสอบถามความพึงพอใจ

ง.1 แบบสอบถามความพึงพอใจ

แบบสอบถามความพึงพอใจ

คำชี้แจง

1. แบบสอบถามความพึงพอใจนี้จัดทำขึ้นเพื่อประเมินประสิทธิภาพของ โปรแกรมจำแนกยุคสมัยประติมากรรมลวดลายปูนปั้นไทย
2. โปรดทำเครื่องหมาย ✓ ลงในช่อง ☐ ที่ตรงกับความเป็นจริงของท่านมากที่สุด

ส่วนที่ 1 ข้อมูลทั่วไป

1.เพศ

☐ ชาย ☐ หญิง ☐ ไม่ระบุ

2.อายุ

☐ ต่ำกว่า 20 ปี ☐ 20 - 40 ปี ☐ 41ปีขึ้นไป

ส่วนที่ 2 ความพึงพอใจต่อการใช้งาน

ความพึงพอใจ	ระดับความพึงพอใจ				
	มากที่สุด (5)	มาก (4)	ปานกลาง (3)	น้อย (2)	น้อยที่สุด (1)
ด้านความสะดวกในการใช้งาน					
1. ความง่ายในการใช้งาน					
2.ประสิทธิภาพการใช้งาน					
3. ฟังก์ชันการใช้งาน					
ด้านการออกแบบหน้าเว็บ					
1.ความสวยงามหน้า UI					
2.ความเข้าใจง่ายของหน้า UI					

ข้อเสนอแนะ

.....

.....

.....

รูปผนวกที่ ง.1 แบบสอบถามความพึงพอใจ